LOW LEVEL DESIGN · CASE STUDY

Splitwise System Design

Study notes that build the full Splitwise LLD in six phases: requirements, UML, design patterns, the greedy *simplify debts* algorithm, and a complete walk-through of the code flow. Every phase has its own folder of runnable Java code.

1 **Users, Groups & Expenses**
entities · composition

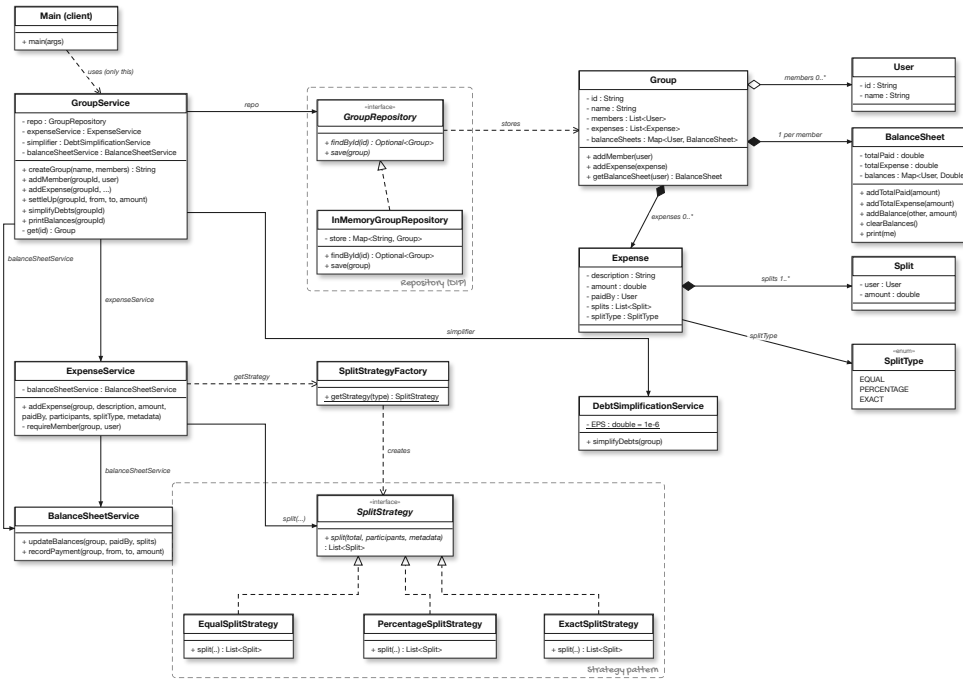
2 **Split Strategy & Factory**
Strategy · Simple Factory · OCP

3 **Balance Sheet & Expense Service**
double-entry ledger · SRP

4 **Repository & Group Service**
Repository · DIP · Facade

5 **Simplify Debts**
greedy · two heaps

6 **Extensions**
exact split · settle up · money



Source. "Splitwise Low Level Design (LLD) Java | Simplify Debts Algorithm, Strategy Pattern & Code" by Shubh Patel (YouTube, 46 min), and its reference code at github.com/shubhkpatel/Low-Level-Design (`splitwise` package).

How to use these notes

We never draw the finished diagram first. We start with the smallest useful piece of the domain and add one idea per phase, always ending with **code that compiles and runs**. Each phase has its own folder with the complete code for that point in the design, so you can run it, diff it against the phase before, or retype it from memory.

In every phase you get

- ✓ **The goal:** which requirement this phase satisfies.
- ✓ **A UML diagram** of everything built so far. Classes added in the phase are tagged NEW and classes that gained members are tagged UPDATED, with the new members shaded.
- ✓ **Concepts and patterns** with code from that phase's folder. In code cards, lines that changed since the previous phase have a purple bar.
- ✓ **Real output** from running the phase.
- ✓ **A checkpoint**: questions to answer before you move on.

Contents

Understand the system: the app, requirements, balance sheet	3
The destination: the video's UML, how to read the diagrams, the pattern toolkit	6
Phase 1 · Users, Groups & Expenses	8
Phase 2 · Split Strategy & Factory	12
Phase 3 · Balance Sheet & Expense Service	15
Phase 4 · Repository & Group Service	20
Phase 5 · Simplify Debts	24
Phase 6 · Extensions	30
The complete code flow (sequence diagrams)	34
The complete design	36
Notes on the reference code · where to take it next	37
Self-test interview questions	38

Folder map

```
Splitwise-LLD-Study/
├── Splitwise_LLD_Study_Notes.pdf   ← this file
├── README.md
├── Phase-1-Users-Groups-and-Expenses/
├── Phase-2-Split-Strategy-and-Factory/
├── Phase-3-Balance-Sheet-and-Expense-Service/
├── Phase-4-Repository-and-Group-Service/
├── Phase-5-Simplify-Debts/      ← the video's design
├── Phase-6-Extensions/          ← final design
├── diagrams/     SVG UML per phase + sequences
└── screenshots/  frames from the video
```

Each phase folder has `run.sh` and `src/splitwise/…`, split into the packages the reference uses: `model`, `enums`, `strategy`, `factory`, `service`, `repository`. You only need a JDK (17+): no Maven, no Lombok.

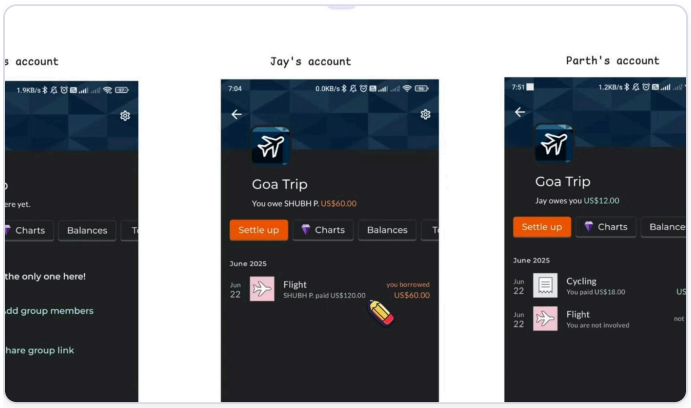
```
bash Phase-3-Balance-Sheet-and-Expense-Service/run.sh
diff -ru Phase-4-Repository-and-Group-Service/src \
         Phase-5-Simplify-Debts/src
```

Study loop that works

Read the phase → **sketch** its UML on paper before looking → **retype** the new classes into a copy of the previous phase (`cp -R Phase-2-… my-phase-3`) → **run** and `diff` against the real folder → answer the **checkpoint**. Retyping teaches more than rereading.

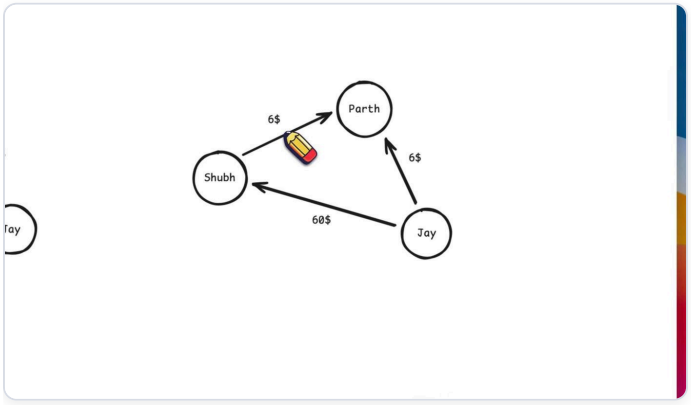
Step 0: Understand the system

Before any class diagram, the video spends seven minutes using the real app with three accounts (Shubh, Jay and Parth). Three behaviours shape the whole design:



Video @ 2:00 · Jay and Parth see the same "Goa Trip" group from their own side.

1. **Groups hold expenses.** Shubh paid $120 for a flight split equally with Jay, so Jay "borrowed $60". Parth sees the flight too, marked *not involved*: an expense names its **participants**, who need not be everyone in the group.
2. **1-on-1 expenses also live in a group.** Every account has a default *Non-group expenses* group. Jay pays a $20 dinner, Shubh pays a $10 ice cream, and the two debts net to "Shubh owes Jay $5". Only pairwise netting happens there, because the people in it may not know each other.
3. **Simplify debts.** Inside a real group, Splitwise can rewrite *who pays whom* so that fewer payments are needed, without changing how much anyone is owed in total.



Video @ 4:05 · flight + cycling, before simplification: 3 payments.

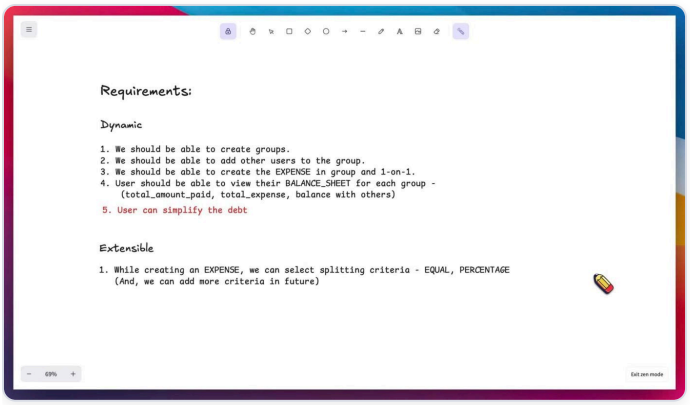
The simplify-debts idea in one example

The flight ($120, Shubh & Jay) and cycling ($18, paid by Parth, split by all three) produce three debts: Jay → Shubh 60, Jay → Parth 6, Shubh → Parth 6.

Look at each person's **net** position instead: Shubh is up 60 − 6 = **+54**, Parth is up **+12**, Jay is down **−66**. Two payments settle that: **Jay → Shubh 54** and **Jay → Parth 12**. That's exactly what the app showed Jay. Phase 5 builds this algorithm, and its demo reproduces these numbers.

> **Interview tip**
> Clarify scope out loud: groups, 1-on-1 expenses, split types, per-group balance sheet, simplify debts. Then say what you are *not* building: payments integration, notifications, currencies, auth.

Requirements



Video @ 10:15 · the requirements board, with item 5 (simplify debts) in red.

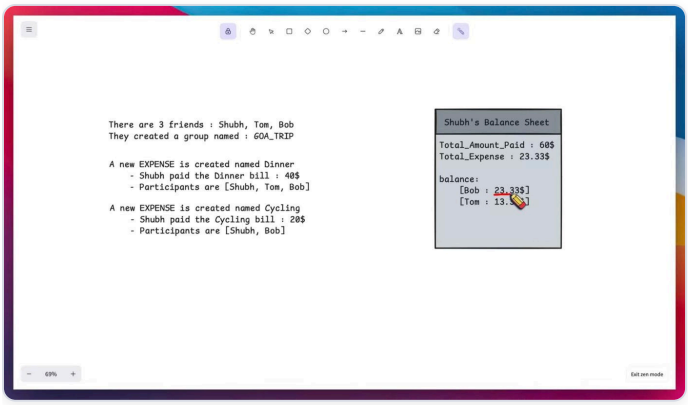
The video splits requirements into **dynamic** ones (what users and admins do at runtime) and **extensible** ones (what must be easy to add later). The extensible one is the hint that a design pattern is expected: "we can add more criteria in future" is a textbook cue for **Strategy**.

From requirement to design decision

#	Requirement (from the video)	Design decision	Phase
D1	Create groups	`Group` entity; `GroupService.createGroup` saves it through a repository	1 4
D2	Add other users to a group	`Group.members : List<User>`; each new member gets an empty `BalanceSheet`	1 3
D3	Create an expense in a group and 1-on-1	`Expense` with `List<Split>`; 1-on-1 = a default "Non-Group Expenses" group	1 4
D4	View my balance sheet per group (total paid, total expense, balance with others)	`BalanceSheet` per user per group, kept up to date by `BalanceSheetService`	3
D5	Simplify the debt	`DebtSimplificationService`: greedy max-creditor ↔ max-debtor matching with two heaps	5
E1	Choose a split criterion (EQUAL, PERCENTAGE); add more later	`SplitStrategy` interface + one class per criterion, picked by `SplitStrategyFactory`	2 6

Why is 1-on-1 just another group?
One code path. A 1-on-1 expense needs exactly what a group expense needs: participants, splits and a balance sheet per person. Modelling it as a hidden default group means no `if (isOneOnOne)` anywhere. The only difference is the app never offers group-wide simplification there.

The balance sheet, before any code



Video @ 9:20 · Shubh's balance sheet after two expenses.

Requirement D4 is the heart of the system, so make sure the numbers are clear. Three friends, one group, two expenses, both paid by Shubh:

- **Dinner $40**, participants Shubh, Tom, Bob → $13.33 each.
- **Cycling $20**, participants Shubh, Bob → $10 each.

Shubh's balance sheet in this group has three parts:

`totalPaid`	money that left Shubh's pocket	40 + 20 = **60**
`totalExpense`	Shubh's own share of everything	13.33 + 10 = **23.33**
`balances`	per other user: + they owe me, − I owe them	Bob **+23.33**, Tom **+13.33**

Two totals, two meanings

`totalPaid` is cash flow; `totalExpense` is consumption. Shubh *paid* 60 but only *consumed* 23.33, so he is owed 60 − 23.33 = 36.67, which is exactly Bob's 23.33 + Tom's 13.33.

The invariant to remember

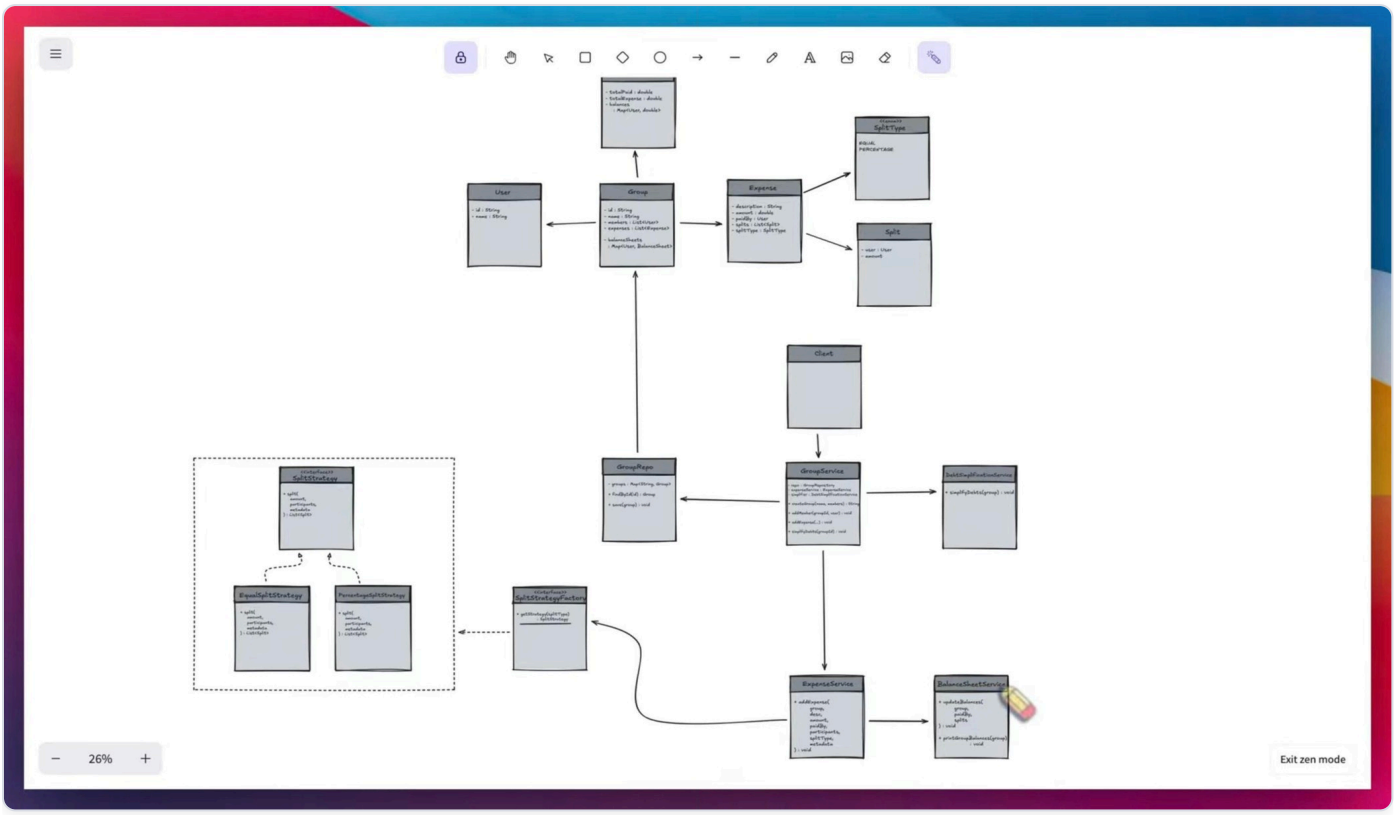
For every user: **totalPaid − totalExpense = sum of balances**. Across the whole group, all balances sum to **0**: every +x on one sheet has a −x on another. Phase 3 tests this.

Say this in the interview

"Each user has one balance sheet per group. Every expense updates two sides: the payer's sheet goes up, each participant's goes down. That symmetry is what makes simplification possible later."

The destination: the UML from the video

This is the whole class diagram the video draws at 19:10, stitched from four zoomed frames. Don't try to memorise it. Phases 1–5 rebuild it piece by piece, and Phase 6 adds a few extensions. The close-ups on each phase's pages come from the same whiteboard.



Video @ 19:10 · models across the top, Client and GroupService in the centre, the strategy family bottom left, ExpenseService and BalanceSheetService along the bottom.

How to read the diagrams in these notes

- **Inheritance** (extends)
- **Realization** (implements an interface)
- **Composition** : the part lives and dies with the whole (Group ◆ Expense)
- **Aggregation** : the part exists on its own (Group ◇ User)
- **Association** : holds a reference in a field
- **Dependency** : uses or creates, but keeps no field

Members: `+` public, `-` private, underlined = static, *italic* = abstract or interface method. «interface» and «enum» are stereotypes.

Your toolkit: the patterns and principles in this design

Strategy 2

Problem: one action (split an amount) has several algorithms, and more will come. **Shape:** an interface `SplitStrategy`, one class per algorithm; the caller holds the interface. **Win:** a new split type never touches existing code.

Simple Factory 2

Problem: the user picks a split type at runtime (an enum value); someone must turn that into an object. **Shape:** `SplitStrategyFactory.getStrategy(SplitType)` with one `switch`. **Win:** the only `switch` on split type lives in one place.

Repository 4

Problem: business logic should not know how groups are stored. **Shape:** `GroupRepository` interface (`findById`, `save`), with `InMemoryGroupRepository` as one implementation. **Win:** swap in SQL later without editing services.

Facade / service layer 4

Problem: the client should not juggle five services. **Shape:** `GroupService` is the only user-facing class; it looks up the group and delegates. **Win:** a small, stable API; internal services can change freely.

Dependency Injection 3 4

Problem: classes that `new` their collaborators are hard to test and swap. **Shape:** collaborators come in through constructors; `Main` wires the graph once. **Win:** pass a fake repository in a test; no code changes.

Greedy with heaps 5

Problem: minimise the number of payments. **Shape:** net balances → max-heap of creditors, min-heap of debtors (most negative first) → match the biggest pair repeatedly. **Win:** *O(N + K log K)*; fast for thousands of users, though not always optimal.

SOLID principles you'll see again and again

Principle	Where it shows up
Single Responsibility	`ExpenseService` creates expenses, `BalanceSheetService` updates ledgers, `DebtSimplificationService` simplifies, `GroupService` only coordinates.
Open/Closed	New split type = new `SplitStrategy` class + one enum value + one factory case (done for real in Phase 6).
Liskov Substitution	Any `SplitStrategy` can stand in for another: same inputs, a `List<Split>` that covers the amount.
Interface Segregation	`SplitStrategy` has one method; `GroupRepository` has only what services use.
Dependency Inversion	`GroupService` depends on the `GroupRepository` interface, not on `InMemoryGroupRepository`.

PHASE
1

Users, Groups & Expenses

Entities · Composition vs aggregation · Enums

Model the nouns first: who is in a group, what was spent, who paid, and who shares the cost. No services, no patterns yet. The client works out every share by hand.

`Phase-1-Users-Groups-and-Expenses/`

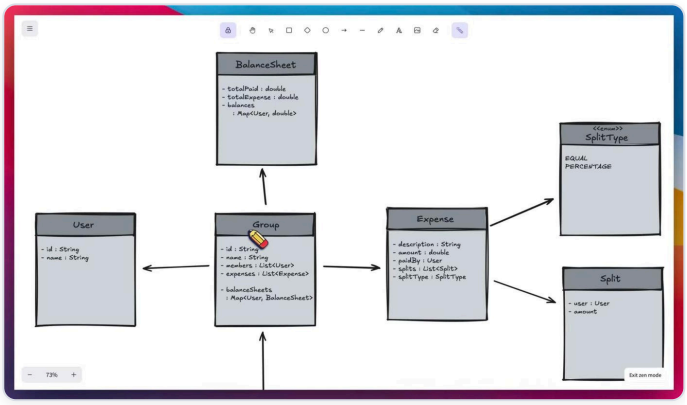
`User` `Group` `Expense` `Split` `SplitType` `Main`

Goal

Requirements **D1–D3** at the data level: a group has members and expenses; an expense knows its amount, its payer, how it was split, and each participant's share.

Concepts

- **Entity vs value.** `User` and `Group` have identity (an `id`). `Split` is a small value: *this user owes this much of this expense*.
- **Composition (◆).** An `Expense` belongs to exactly one `Group`, and a `Split` to one `Expense`. Delete the group and its expenses go too.
- **Aggregation (◇).** A `User` exists outside any group and can be in many, so `Group` only *refers* to users.
- **Enum for the split type.** `SplitType` is a closed set of choices the user picks from. It is data, not behaviour; behaviour arrives in Phase 2.

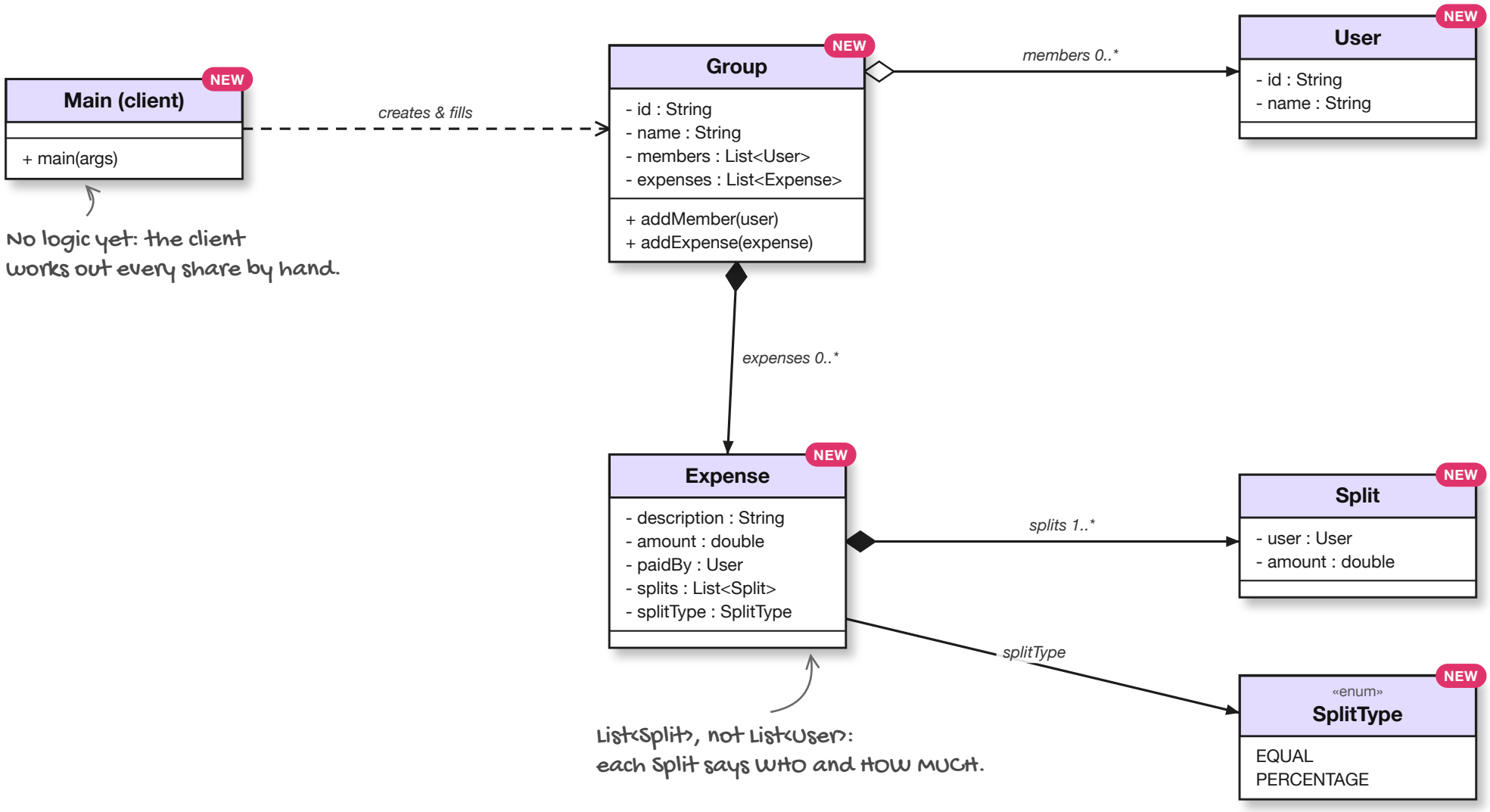


Video @ 11:40 · the model classes on the whiteboard.

The design decision the video calls out: List<Split>, not List<User>

The first draft gave `Expense` a `List<User> participants`. That only says *who*. With a percentage split everyone pays a different amount, so you need *who* and *how much* together. A `Split(user, amount)` carries both, and the participant list is just `splits.map(Split::getUser)`.

Phase 1 UML: the domain model



New: `User`, `Group`, `Expense`, `Split`, `SplitType`. Read it as: a Group ◇ refers to its members, ◆ owns its expenses; an Expense ◆ owns its splits and names a SplitType.

model/Split.java NEW

```java
// One participant's share of an expense: "who" plus "how much".
public class Split {
    private final User user;
    private double amount;

    public Split(User user, double amount) {
        this.user = user;
        this.amount = amount;
    }

    public User getUser() { return user; }
    public double getAmount() { return amount; }
    public void setAmount(double amount) { this.amount = amount; }

    @Override
    public String toString() { return String.format("%s=%.2f", user.getName(), amount); }
}
```

enums/SplitType.java NEW

```java
public enum SplitType {
    EQUAL, PERCENTAGE
}
```

model/User.java NEW

```java
public class User {
    private final String id;
    private final String name;

    public User(String id, String name) {
        this.id = id;
        this.name = name;
    }

    public String getId() { return id; }
    public String getName() { return name; }
```

model/Group.java NEW

```java
public class Group {
    private final String id;
    private final String name;
    private final List<User> members = new ArrayList<>();
    private final List<Expense> expenses = new ArrayList<>
     ();

    public Group(String id, String name) {
        this.id = id;
        this.name = name;
    }

    public void addMember(User user) {
        members.add(user);
    }

    public void addExpense(Expense expense) {
        expenses.add(expense);
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public List<User> getMembers() { return members; }
    public List<Expense> getExpenses() { return expenses; }
}
```

model/Expense.java NEW

```java
public class Expense {
    private final String description;
    private final double amount;
    private final User paidBy;
    private final List<Split> splits;
    private final SplitType splitType;

    public Expense(String description, double amount, User paidBy, List<Split> splits, SplitType splitType) {
        this.description = description;
        this.amount = amount;
        this.paidBy = paidBy;
        this.splits = splits;
        this.splitType = splitType;
    }
```

Main.java NEW

```java
        Group goa = new Group("g1", "Goa Trip");
        List.of(shubh, bob, tom).forEach(goa::addMember);

        // No strategy yet: we work out each share by hand.
        goa.addExpense(new Expense("Lunch Day-1", 100, shubh,
                List.of(new Split(shubh, 50), new Split(bob, 50)), SplitType.EQUAL));
        goa.addExpense(new Expense("Cab", 90, tom,
                List.of(new Split(shubh, 45), new Split(bob, 27), new Split(tom, 18)), SplitType.PERCENTAGE));

        System.out.println("Group: " + goa.getName() + " " + goa.getMembers());
        goa.getExpenses().forEach(e -> System.out.println("  " + e));
```

```
▶ bash Phase-1-Users-Groups-and-Expenses/run.sh

═══ Phase 1: Users, Groups & Expenses ═══
Group: Goa Trip [Shubh, Bob, Tom]
  Lunch Day-1: 100.00 paid by Shubh, EQUAL split [Shubh=50.00, Bob=50.00]
  Cab: 90.00 paid by Tom, PERCENTAGE split [Shubh=45.00, Bob=27.00, Tom=18.00]
```

Why is Expense immutable but Split has a setter?

An expense, once recorded, is a historical fact, so every field is `final`. The reference gives `Split` a setter for `amount`, presumably so a strategy can adjust a share (say, a rounding cent) after creating it. Nothing calls it yet.

Checkpoint

1) Why does `Group` aggregate `User` but compose `Expense`?

2) The "Cab" expense says PERCENTAGE, but who computed 45 / 27 / 18? *(The client did, by hand. Phase 2 moves that into a strategy.)*

PHASE 2

Split Strategy & Factory

Strategy · Simple Factory · Open/Closed

Turn "how do we split this bill?" into interchangeable objects. The caller names a SplitType; a factory hands back the right algorithm; the algorithm returns the List<Split>.

Phase-2-Split-Strategy-and-Factory/

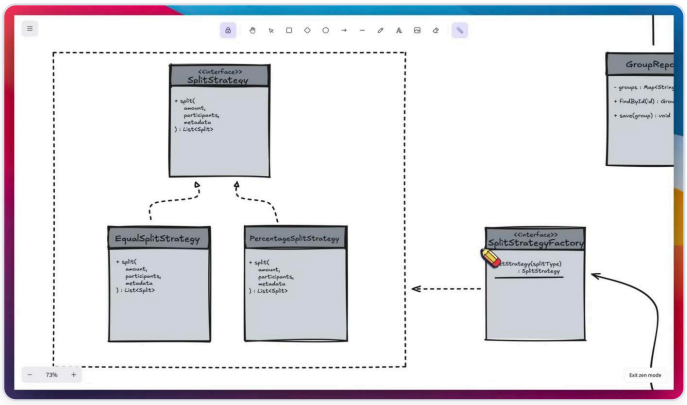
`SplitStrategy` `EqualSplitStrategy` `PercentageSplitStrategy` `SplitStrategyFactory`

Goal

Requirement **E1**: choose EQUAL or PERCENTAGE when creating an expense, and be able to add more split types later without editing the code that creates expenses.

The Strategy pattern, step by step

1. Find what varies: *how an amount becomes shares*.
2. Put it behind an interface: `split(total, participants, metadata) → List<Split>`.
3. One class per algorithm: `EqualSplitStrategy`, `PercentageSplitStrategy`.
4. The context (the client now, `ExpenseService` from Phase 3) holds only a `SplitStrategy` and calls `split`.
5. A **factory** maps the user's `SplitType` choice to a strategy object at runtime.



Video @ 16:40 · the strategy family in a dashed box, with the factory pointing at it.

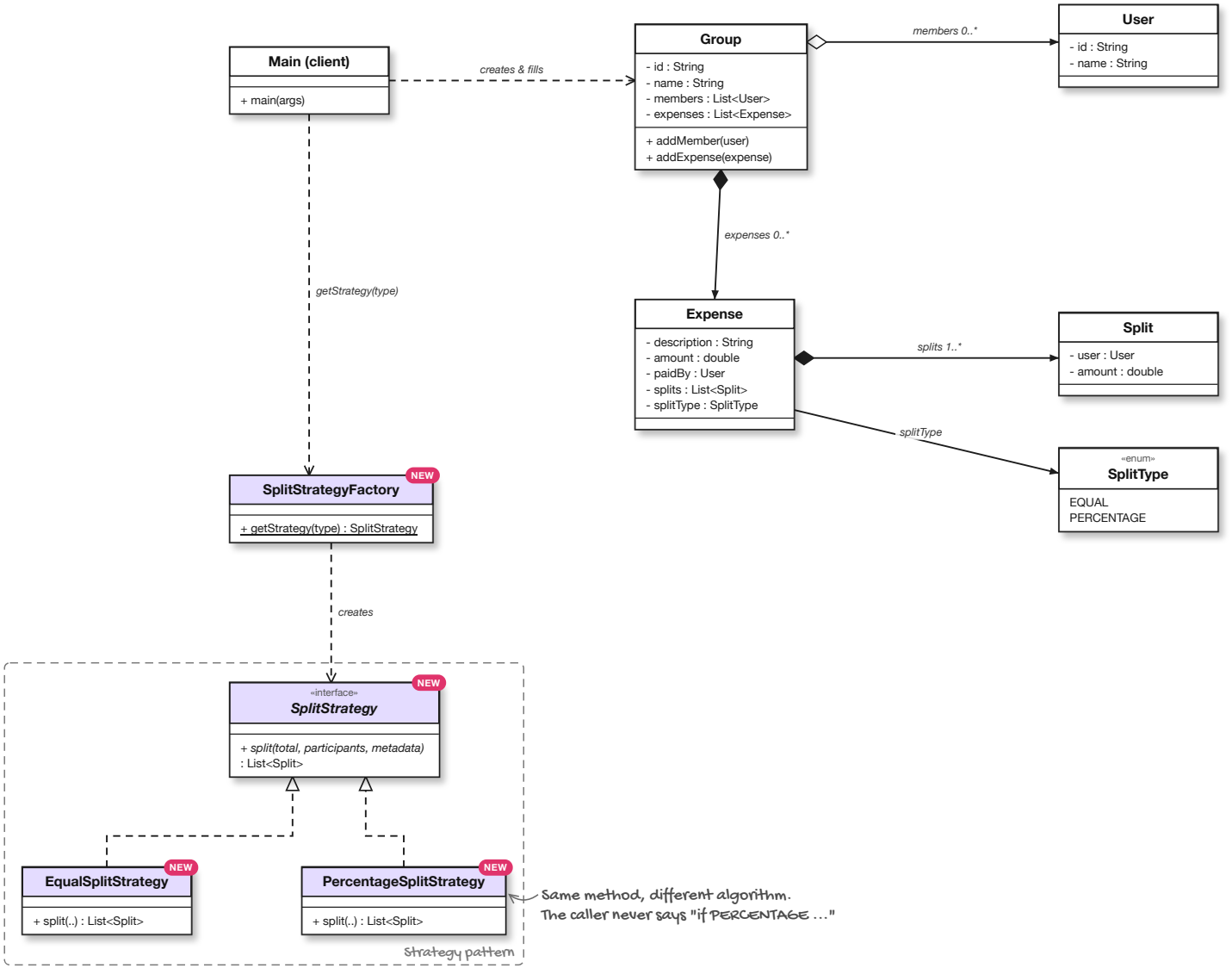
What is `metadata` for?

Different algorithms need different extra input. EQUAL needs nothing; PERCENTAGE needs *user → percent*; an exact split needs *user → amount*. A single `Map<User, Double>` keeps one method signature for all of them. The trade-off: the type system no longer says what each strategy expects, so each strategy must validate its own metadata (Phase 6 does).

Strategy vs Factory in one sentence each

Strategy lets me add more splitting algorithms while keeping the interface intact. **Factory** lets me create the right strategy object at runtime from the user's input. (Paraphrasing the video at 17:00.)

Phase 2 UML: the strategy family plugs in



New: `SplitStrategy` and two implementations (dashed arrows = *implements*), plus `SplitStrategyFactory`, which *creates* them. The domain model is unchanged.

strategy/SplitStrategy.java NEW

```java
public interface SplitStrategy {
    // metadata carries per-user inputs a strategy may need (e.g. percentages); EQUAL ignores it.
    List<Split> split(double totalAmount, List<User> participants, Map<User, Double> metadata);
}
```

strategy/EqualSplitStrategy.java NEW

```java
public class EqualSplitStrategy implements SplitStrategy {
    @Override
    public List<Split> split(double totalAmount, List<User> participants, Map<User, Double> metadata) {
        double share = totalAmount / participants.size();
        List<Split> splits = new ArrayList<>();
        for (User user : participants) {
            splits.add(new Split(user, share));
        }
        return splits;
    }
}
```

factory/SplitStrategyFactory.java NEW

```java
public class SplitStrategyFactory {
    public static SplitStrategy getStrategy(SplitType
     splitType) {
        return switch (splitType) {
            case EQUAL -> new EqualSplitStrategy();
            case PERCENTAGE -> new
     PercentageSplitStrategy();
        };
    }
}
```

Why a `switch` expression in the factory?

Java's `switch` expression over an enum must be **exhaustive**. Add `EXACT` to `SplitType` and forget the factory case, and the code no longer compiles. The compiler points you at the one place to update.

Checkpoint

1) Where is the only `if/switch` on split type in the whole program?
2) Equal split of 18 among 3 gives 6.00 each, but what does 100 among 3 give? *(33.333..., which never adds back to 100.00 in cents. Phase 6 fixes it.)*
3) Why does PercentageSplitStrategy throw instead of returning an empty list?

strategy/PercentageSplitStrategy.java NEW

```java
public class PercentageSplitStrategy implements SplitStrategy {
    @Override
    public List<Split> split(double totalAmount, List<User> participants, Map<User, Double> metadata) {
        double totalPercent = metadata.values().stream().mapToDouble(Double::doubleValue).sum();
        if (totalPercent != 100.0) throw new IllegalArgumentException("Total percent should be 100");

        List<Split> splits = new ArrayList<>();
        for (User user : participants) {
            splits.add(new Split(user, totalAmount * metadata.getOrDefault(user, 0.0) / 100));
        }
        return splits;
    }
}
```

Main.java (excerpt): the client uses the pattern CHANGED

```java
    private static Expense createExpense(String description, double amount, User paidBy,
                                         List<User> participants, SplitType type, Map<User, Double> metadata) {
        SplitStrategy strategy = SplitStrategyFactory.getStrategy(type);
        List<Split> splits = strategy.split(amount, participants, metadata);
        return new Expense(description, amount, paidBy, splits, type);
    }
```

```
▶ bash Phase-2-Split-Strategy-and-Factory/run.sh

═══ Phase 2: Split Strategy & Factory ═══
Group: Goa Trip [Shubh, Bob, Tom]
  Lunch Day-1: 100.00 paid by Shubh, EQUAL split [Shubh=50.00, Bob=50.00]
  Cab: 90.00 paid by Tom, PERCENTAGE split [Shubh=45.00, Bob=27.00, Tom=18.00]
  Cycling: 18.00 paid by Shubh, EQUAL split [Shubh=6.00, Bob=6.00, Tom=6.00]
  Rejected 60% + 30%: Total percent should be 100
```

PHASE

3

Balance Sheet & Expense Service

Double-entry ledger · Single Responsibility · Constructor injection

Give every member a ledger, and move "create an expense" out of the client into a service that splits the bill, records the expense and updates both sides of every debt.

`Phase-3-Balance-Sheet-and-Expense-Service/`

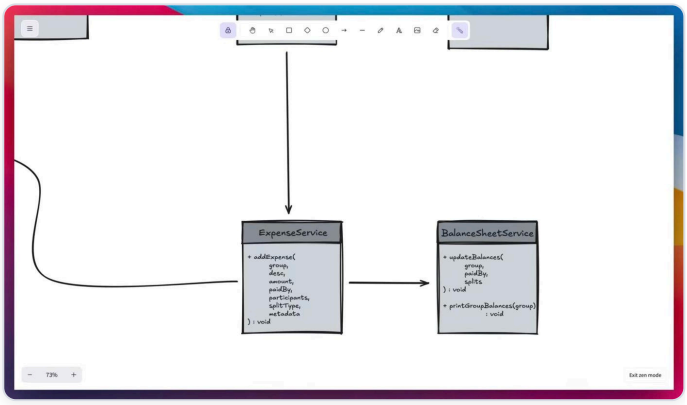
`BalanceSheet` `BalanceSheetService` `ExpenseService` `Group (updated)`

Goal

Requirement **D4**: each user can see total paid, total expense and the balance with every other member, per group.

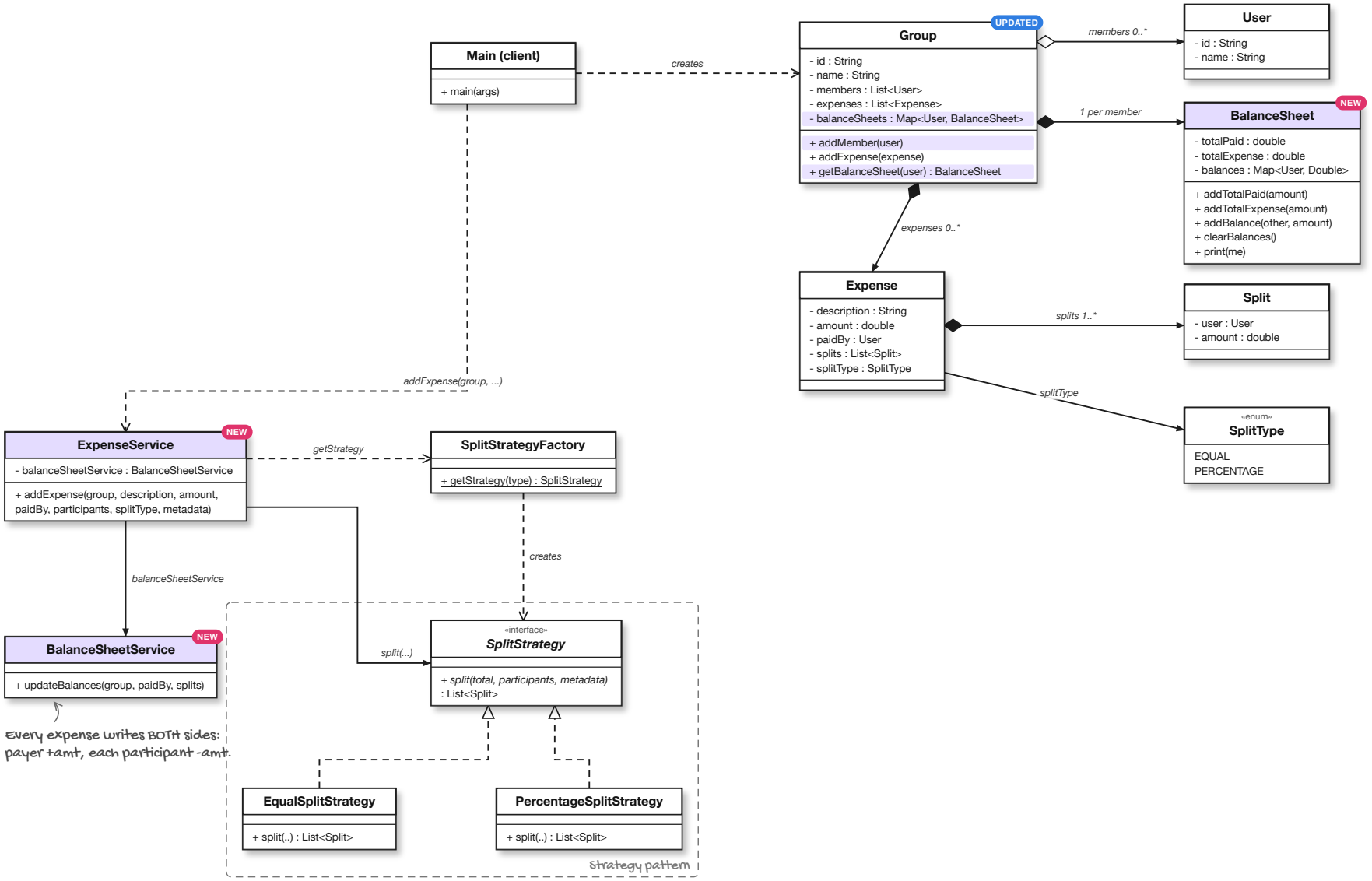
Concepts

- **One sheet per (user, group).** `Group` now holds `Map<User, BalanceSheet>`, and `addMember` creates the empty sheet.
- **Two-sided updates.** For each participant who isn't the payer: participant's sheet gets `-share` against the payer, the payer's sheet gets `+share` against the participant.
- **Pairwise netting for free.** `addBalance` *adds* to the existing entry, so A owing B 10 and B owing A 4 collapse to one entry of 6. An entry that hits 0 is removed.
- **SRP.** `ExpenseService` orchestrates (strategy → expense → ledger); `BalanceSheetService` owns the ledger arithmetic.



Video @ 15:30 · ExpenseService delegates the ledger update to BalanceSheetService.

Phase 3 UML: ledgers and the first services



New: `BalanceSheet` (one per member, owned by Group), `BalanceSheetService` and `ExpenseService`. The client now calls `ExpenseService` instead of using the factory itself.

model/BalanceSheet.java — NEW

```java
// One user's ledger inside one group.
public class BalanceSheet {
    private double totalPaid = 0.0;      // money this user actually paid out
    private double totalExpense = 0.0;   // this user's own share of all expenses
    // other user → amount. Positive: they owe me. Negative: I owe them.
    // LinkedHashMap (not HashMap) only so the printed order is stable run to run.
    private final Map<User, Double> balances = new LinkedHashMap<>();

    public void addTotalPaid(double amount) {
        this.totalPaid += amount;
    }

    public void addTotalExpense(double amount) {
        this.totalExpense += amount;
    }

    public void addBalance(User other, double amount) {
        balances.put(other, balances.getOrDefault(other, 0.0) + amount);
        if (Math.abs(balances.get(other)) < 1e-6) balances.remove(other);
    }

    public void clearBalances() {
        balances.clear();
    }

    public double getTotalPaid() { return totalPaid; }
    public double getTotalExpense() { return totalExpense; }
    public Map<User, Double> getBalances() { return balances; }
```

model/Group.java — CHANGED

```java
public class Group {
    private final String id;
    private final String name;
    private final List<User> members = new ArrayList<>();
    private final List<Expense> expenses = new ArrayList<>();
    private final Map<User, BalanceSheet> balanceSheets = new HashMap<>();

    public Group(String id, String name) {
        this.id = id;
        this.name = name;
    }

    public void addMember(User user) {
        members.add(user);
        balanceSheets.putIfAbsent(user, new BalanceSheet());
    }

    public void addExpense(Expense expense) {
        expenses.add(expense);
    }

    public BalanceSheet getBalanceSheet(User user) {
        return balanceSheets.get(user);
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public List<User> getMembers() { return members; }
    public List<Expense> getExpenses() { return expenses; }
    public Map<User, BalanceSheet> getBalanceSheets() { return balanceSheets; }
}
```

service/BalanceSheetService.java NEW

```java
public class BalanceSheetService {

    public void updateBalances(Group group, User paidBy, List<Split> splits) {
        double totalAmount = splits.stream().mapToDouble(Split::getAmount).sum();
        group.getBalanceSheet(paidBy).addTotalPaid(totalAmount);

        for (Split split : splits) {
            User user = split.getUser();
            double amt = split.getAmount();
            group.getBalanceSheet(user).addTotalExpense(amt);
            if (!user.equals(paidBy)) {
                // Two-sided entry: the participant owes the payer, the payer is owed by the participant.
                group.getBalanceSheet(user).addBalance(paidBy, -amt);
                group.getBalanceSheet(paidBy).addBalance(user, amt);
            }
        }
    }
}
```

service/ExpenseService.java NEW

```java
public class ExpenseService {
    private final BalanceSheetService balanceSheetService;

    public ExpenseService(BalanceSheetService balanceSheetService) {
        this.balanceSheetService = balanceSheetService;
    }

    public void addExpense(Group group, String description, double amount, User paidBy,
                           List<User> participants, SplitType splitType, Map<User, Double> metadata) {

        SplitStrategy strategy = SplitStrategyFactory.getStrategy(splitType);
        List<Split> splits = strategy.split(amount, participants, metadata);
        Expense expense = new Expense(description, amount, paidBy, splits, splitType);
        group.addExpense(expense);

        balanceSheetService.updateBalances(group, paidBy, splits);
    }
}
```

Trace: what `updateBalances` writes for the video's example

Expense	Split	Shubh (payer)	Tom	Bob
Dinner 40	Shubh 13.33	paid +40, expense +13.33		
	Tom 13.33	Tom: **+13.33**	expense +13.33, Shubh: **−13.33**	
	Bob 13.33	Bob: **+13.33**		expense +13.33, Shubh: **−13.33**
Cycling 20	Shubh 10	paid +20, expense +10		
	Bob 10	Bob: **+10 → 23.33**		expense +10, Shubh: **−10 → −23.33**
Final sheet		**paid 60, expense 23.33, Tom +13.33, Bob +23.33**	**paid 0, expense 13.33, Shubh −13.33**	**paid 0, expense 23.33, Shubh −23.33**

```
▶ bash Phase-3-Balance-Sheet-and-Expense-Service/run.sh

═══ Phase 3: Balance Sheet & Expense Service ═══
  Dinner: 40.00 paid by Shubh, EQUAL split [Shubh=13.33, Tom=13.33, Bob=13.33]
  Cycling: 20.00 paid by Shubh, EQUAL split [Shubh=10.00, Bob=10.00]

Balance sheet of Shubh
  Paid: 60.00  Expense: 23.33  Owe: 0.00  Get back: 36.67
    gets 13.33 from Tom
    gets 23.33 from Bob
Balance sheet of Tom
  Paid: 0.00  Expense: 13.33  Owe: 13.33  Get back: 0.00
    owes 13.33 to Shubh
Balance sheet of Bob
  Paid: 0.00  Expense: 23.33  Owe: 23.33  Get back: 0.00
    owes 23.33 to Shubh
```

The numbers match the whiteboard example exactly. Check the invariant: Shubh 60 − 23.33 = 36.67 = 13.33 + 23.33; Tom 0 − 13.33 = −13.33; Bob 0 − 23.33 = −23.33; and 36.67 − 13.33 − 23.33 = 0 across the group.

Why not store one "who owes whom" list on the Group?

You could, and the simplifier would like it. But the requirement is a *per-user* view, and each user's sheet answers "what do I owe and what am I owed?" with a single map lookup. The price is that every debt is written twice; `BalanceSheetService` exists so that is done in exactly one place.

Watch out: User has no equals()/hashCode()

Balance maps are keyed by `User` objects, so they compare by **identity**. That works here because each person is one object for the whole run. Load users from a database and you get two `User` objects for the same id, and the maps break. Real fix: implement `equals` / `hashCode` on `id`. (`LinkedHashMap` is only there to make the printed order stable.)

Checkpoint

1) After Lunch Day-1 (Shubh pays 100 for Shubh and Bob) and Lunch Day-2 (Bob pays 100 for Bob and Tom), what is on Bob's sheet? *(paid 100, expense 100, Shubh −50, Tom +50)*
2) Which class would you change to also email people when they are owed money? Which would you leave alone?

PHASE 4

Repository & Group Service

Repository · Dependency Inversion · Facade

Hide storage behind an interface and give the client a single door into the system. From now on Main only talks to GroupService, using group ids.

Phase-4-Repository-and-Group-Service/

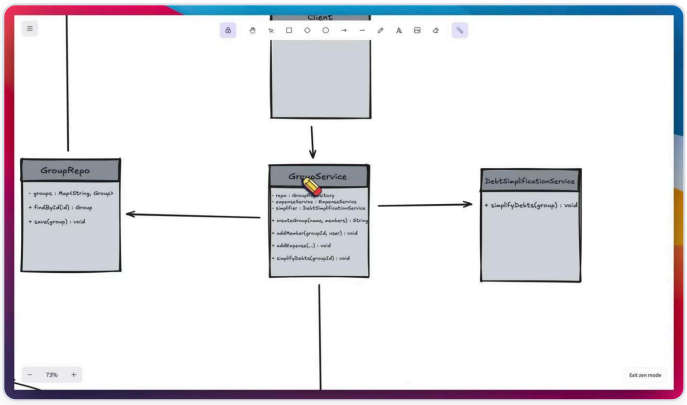
`GroupRepository` `InMemoryGroupRepository` `GroupService`

Goal

Requirements **D1–D3** at the API level: create a group, add members, add group and 1-on-1 expenses, all through one service, with groups stored somewhere the services don't care about.

Concepts

- **Repository.** A collection-like interface for loading and saving aggregates. Services ask for "group `g7`", not "row 7 of table groups".
- **Dependency Inversion.** High-level `GroupService` and low-level `InMemoryGroupRepository` both depend on the `GroupRepository` abstraction.
- **Facade.** `GroupService` is user-facing; `ExpenseService` and `BalanceSheetService` are internal.
- **Composition root.** `Main` builds the object graph once and passes dependencies into constructors.



Video @ 14:30 · Client → GroupService → GroupRepo / ExpenseService / DebtSimplificationService.

DIP: the before and after the video describes (28:30)

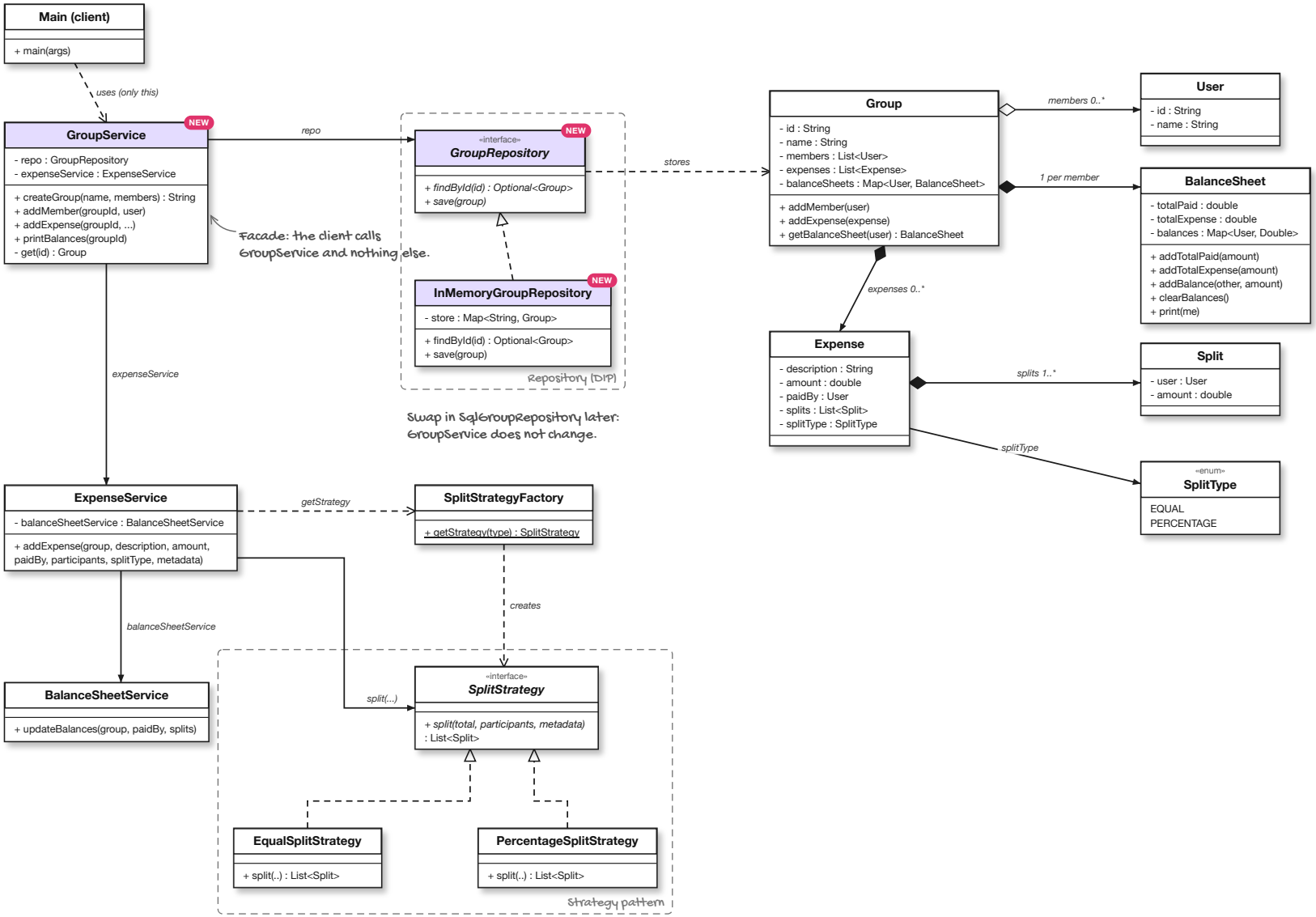
Without the interface ✗

```
public class GroupService {
    // Depends on a concrete class:
    private final InMemoryGroupRepository repo = new
     InMemoryGroupRepository();
    // Moving to SQL means editing this class (and every
     test of it).
}
```

With the interface ✓

```
public class GroupService {
    private final GroupRepository repo;          // the
     abstraction
    public GroupService(GroupRepository repo, ...) {
        this.repo = repo;                         //
     InMemory today, Sql tomorrow
    }
}
```

Phase 4 UML: one door in, storage behind an interface



New: `GroupService` (facade), `GroupRepository` and `InMemoryGroupRepository`. The client moved to the top-left: it only knows `GroupService`.

repository/GroupRepository.java NEW

```java
public interface GroupRepository {
    Optional<Group> findById(String id);
    void save(Group group);
}
```

repository/InMemoryGroupRepository.java NEW

```java
public class InMemoryGroupRepository implements GroupRepository {

    private final Map<String, Group> store = new HashMap<>();

    @Override
    public Optional<Group> findById(String id) { return Optional.ofNullable(store.get(id)); }

    @Override
    public void save(Group g) { store.put(g.getId(), g); }
}
```

service/GroupService.java NEW

```java
// The only service the client talks to. It owns no business rules itself; it looks up the
// group and hands the work to the right collaborator.
public class GroupService {

    private final GroupRepository repo;
    private final ExpenseService expenseService;

    public GroupService(GroupRepository repo, ExpenseService expenseService) {
        this.repo = repo;
        this.expenseService = expenseService;
    }

    public String createGroup(String name, List<User> members) {
        String id = UUID.randomUUID().toString();
        Group g = new Group(id, name);
        members.forEach(g::addMember);

        repo.save(g);
        return id;
    }

    public void addMember(String groupId, User user) {
        get(groupId).addMember(user);
    }

    public void addExpense(String groupId,
                           String description, double amount,
                           User paidBy, List<User> participants,
                           SplitType splitType, Map<User, Double> meta) {

        expenseService.addExpense(get(groupId), description, amount,
                paidBy, participants, splitType, meta);
    }

    public void printBalances(String groupId) {
        Group g = get(groupId);
        System.out.println("[" + g.getName() + "]");
        g.getMembers().forEach(u -> g.getBalanceSheet(u).print(u));
    }

    private Group get(String id) {
        return repo.findById(id)
                .orElseThrow(() -> new IllegalArgumentException("Group not found: " + id));
    }
}
```

Main.java (excerpt): wiring once, then GroupService only — CHANGED

```java
        // Wiring: build the object graph once, inject dependencies through constructors.
        InMemoryGroupRepository repo = new InMemoryGroupRepository();
        BalanceSheetService balanceSheetService = new BalanceSheetService();
        ExpenseService expenseService = new ExpenseService(balanceSheetService);
        GroupService groupService = new GroupService(repo, expenseService);

        // From here on the client only talks to GroupService.
        String goa = groupService.createGroup("Goa Trip", List.of(shubh, bob, tom));
        groupService.addExpense(goa, "Lunch Day-1", 100, shubh, List.of(shubh, bob), SplitType.EQUAL, null);
        groupService.addExpense(goa, "Lunch Day-2", 100, bob, List.of(bob, tom), SplitType.EQUAL, null);
        groupService.printBalances(goa);

        // 1-on-1 expenses live in a default "Non-Group Expenses" group.
        String nonGroup = groupService.createGroup("Non-Group Expenses", List.of(shubh, jay));
        groupService.addExpense(nonGroup, "Dinner", 20, jay, List.of(shubh, jay), SplitType.EQUAL, null);
        groupService.addExpense(nonGroup, "Ice cream", 10, shubh, List.of(shubh, jay), SplitType.EQUAL, null);
        System.out.println();
        groupService.printBalances(nonGroup);
```

```
▶ bash Phase-4-Repository-and-Group-Service/run.sh

═══ Phase 4: Repository & Group Service ═══
[Goa Trip]
Balance sheet of Shubh
  Paid: 100.00  Expense: 50.00  Owe: 0.00  Get back: 50.00
    gets 50.00 from Bob
Balance sheet of Bob
  Paid: 100.00  Expense: 100.00  Owe: 50.00  Get back: 50.00
    owes 50.00 to Shubh
    gets 50.00 from Tom
Balance sheet of Tom
  Paid: 0.00  Expense: 50.00  Owe: 50.00  Get back: 0.00
    owes 50.00 to Bob

[Non-Group Expenses]
Balance sheet of Shubh
  Paid: 10.00  Expense: 15.00  Owe: 5.00  Get back: 0.00
    owes 5.00 to Jay
Balance sheet of Jay
  Paid: 20.00  Expense: 15.00  Owe: 0.00  Get back: 5.00
    gets 5.00 from Shubh

Rejected: Group not found: no-such-id
```

Why does GroupService take ids, not Group objects?

The client (a controller, an app screen) only knows ids from a URL or a tap. Looking the group up is the service's job, which is why `get(id)` uses `Optional.orElseThrow`: an unknown id fails fast with a clear message instead of a `NullPointerException` three calls later.

Checkpoint

1) Write the header of a `SqlGroupRepository`. Which other files change? *(Only Main, where it is wired in.)*
2) In the 1-on-1 group, why is Shubh's total expense 15 but his balance only −5?
3) Is `GroupService` a Facade, a Service, or both?

PHASE 5

Simplify Debts

Graph thinking · Greedy · Two heaps

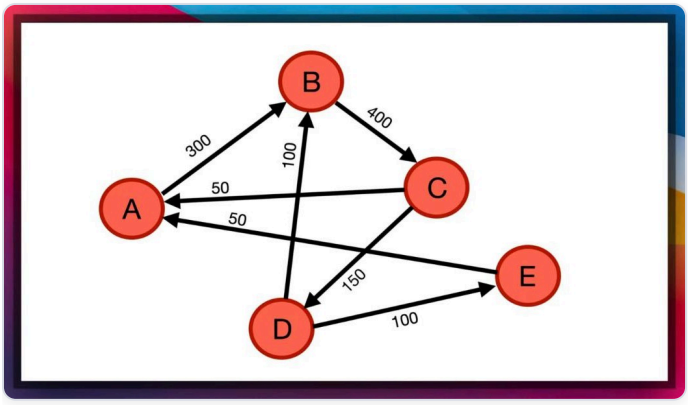
Rewrite who pays whom so the group needs fewer payments, while every person still ends up paying or receiving exactly the same net amount. This is the video's complete design.

Phase-5-Simplify-Debts/

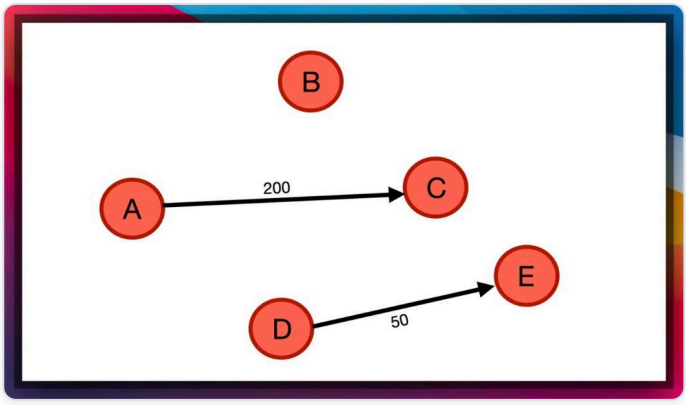
`DebtSimplificationService` `GroupService (updated)`

Goal

Requirement **D5**. Model the group as a graph: **nodes** are users and a **directed edge** X → Y with weight w means "X owes Y w". Simplification keeps every node's net position but rebuilds the edges so there are as few as possible.



Video @ 37:48 · the test graph: 7 transactions.



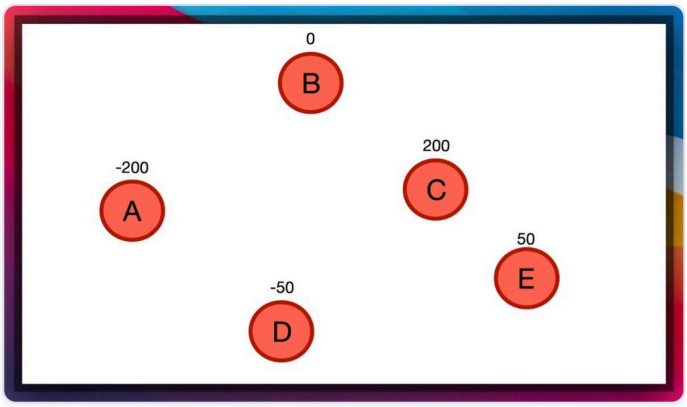
Video @ 38:42 · after simplification: 2 transactions. A→C is a brand-new edge.

Step 1: net balances (throw the edges away)

For each user, net = what they receive − what they pay. Edges only matter through these totals.

A	−300 (to B) + 50 (from C) + 50 (from E)	**−200**
B	+300 + 100 − 400	**0**
C	+400 − 50 − 150	**+200**
D	+150 − 100 − 100	**−50**
E	+100 − 50	**+50**

Nets always sum to 0 (here −200 + 0 + 200 − 50 + 50). B is settled and drops out.



Video @ 40:12 · every node now carries only its net.

Step 2: senders and receivers

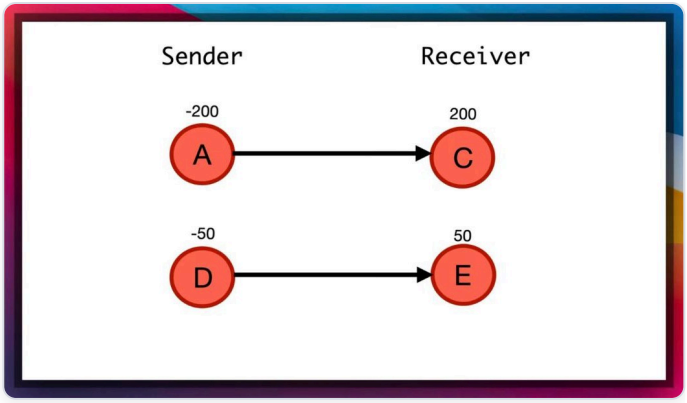
Negative nets are **senders** (debtors); positive nets are **receivers** (creditors). Sending money from one sender to another sender never helps, so every payment goes sender → receiver.

Step 3: the greedy rule

The video weighs sender A's three options: pay D (another sender, useless), pay E (only clears 50, so A needs a second payment), or pay C (clears all 200 in one go). The rule that falls out:

> Take the **biggest debtor** and the **biggest creditor**; settle `min(debt, credit)`; put back whoever isn't at zero; repeat.

Two priority queues make "biggest" O(log K): a **max-heap of creditors** by net, and a **min-heap of debtors** by net (most negative first).



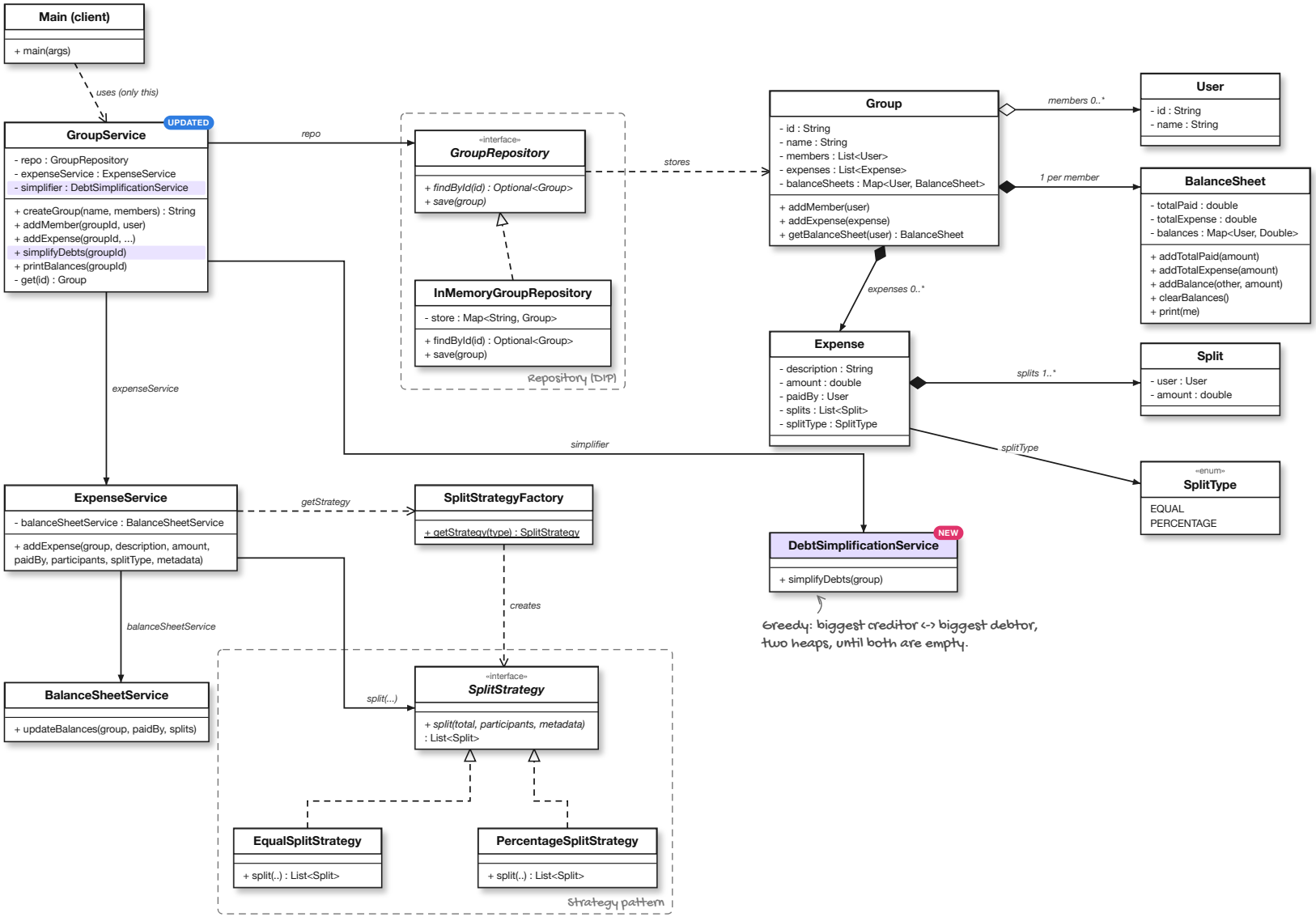
Video @ 43:20 · the greedy result: A→C 200, D→E 50.

Greedy trace for the A–E graph

Turn	creditors (max-heap)	debtors (min-heap)	poll	settle	payment	put back
1	C 200, E 50	A −200, D −50	C, A	min(200, 200) = 200	**A → C 200**	none
2	E 50	D −50	E, D	min(50, 50) = 50	**D → E 50**	none

Both heaps empty → 2 payments instead of 7. Each turn settles at least one person fully, so K non-zero people need at most K − 1 payments.

Phase 5 UML: the video's complete design



New: `DebtSimplificationService`, reached only through `GroupService.simplifyDebts`. This is the whiteboard design from the video, class for class.

service/DebtSimplificationService.java NEW

```java
public class DebtSimplificationService {

    public void simplifyDebts(Group group) {
        List<User> users = new ArrayList<>(group.getMembers());
        Map<User, BalanceSheet> sheets = group.getBalanceSheets();

        // Step 1: Calculate net balances for each user
        Map<User, Double> netBalances = new HashMap<>();
        for (User user : users) {
            double net = 0.0;
            Map<User, Double> balances = sheets.get(user).getBalances();
            for (double amount : balances.values()) {
                net += amount;
            }
            netBalances.put(user, net);
            sheets.get(user).clearBalances(); // Clear old balances before recomputing
        }

        // Step 2: Separate creditors and debtors
        PriorityQueue<User> creditors = new PriorityQueue<>((a, b) -> Double.compare(netBalances.get(b),
     netBalances.get(a)));
        PriorityQueue<User> debtors = new PriorityQueue<>((a, b) -> Double.compare(netBalances.get(a), netBalances.get(b)));

        for (User user : users) {
            double net = netBalances.get(user);
            if (net > 0) {
                creditors.offer(user);
            } else if (net < 0) {
                debtors.offer(user);
            }
        }

        // Step 3: Match debtors and creditors to settle debts
        while (!creditors.isEmpty() && !debtors.isEmpty()) {
            User creditor = creditors.poll();
            User debtor = debtors.poll();

            double creditAmount = netBalances.get(creditor);
            double debitAmount = netBalances.get(debtor);

            double settledAmount = Math.min(creditAmount, -debitAmount);

            // Update balances both sides
            sheets.get(creditor).addBalance(debtor, settledAmount);
            sheets.get(debtor).addBalance(creditor, -settledAmount);

            // Update net balances after settlement
            netBalances.put(creditor, creditAmount - settledAmount);
            netBalances.put(debtor, debitAmount + settledAmount);

            // If still unsettled, re-add to queues
            if (netBalances.get(creditor) > 0) {
                creditors.offer(creditor);
            }
            if (netBalances.get(debtor) < 0) {
                debtors.offer(debtor);
            }
        }
    }
}
```

Step 1 clears every sheet while computing nets, Step 3 writes the new, smaller set of debts back through `addBalance` on both sides. `totalPaid` and `totalExpense` are never touched: simplifying changes *who pays whom*, not *what anyone spent*.

service/GroupService.java (changes) CHANGED

```java
    private final DebtSimplificationService simplifier;

    public GroupService(GroupRepository repo, ExpenseService expenseService, DebtSimplificationService simplifier) {
        this.repo = repo;
        this.expenseService = expenseService;
        this.simplifier = simplifier;
    }

    public String createGroup(String name, List<User> members) {
        String id = UUID.randomUUID().toString();
        Group g = new Group(id, name);
        members.forEach(g::addMember);

        repo.save(g);
        return id;
    }

    public void addMember(String groupId, User user) {
        get(groupId).addMember(user);
    }

    public void addExpense(String groupId,
                           String description, double amount,
                           User paidBy, List<User> participants,
                           SplitType splitType, Map<User, Double> meta) {

        expenseService.addExpense(get(groupId), description, amount,
                paidBy, participants, splitType, meta);
    }

    public void simplifyDebts(String groupId) {
        simplifier.simplifyDebts(get(groupId));
```

Main.java (excerpt): "X owes Y" as an expense NEW

```java
    // "debtor owes creditor amount" is just an expense the creditor paid for the debtor alone.
    private static void owes(String groupId, User debtor, User creditor, double amount) {
        groupService.addExpense(groupId, debtor.getName() + " owes " + creditor.getName(),
                amount, creditor, List.of(debtor), SplitType.EQUAL, null);
    }
```

```
▶ bash Phase-5-Simplify-Debts/run.sh

═══ Phase 5: Simplify Debts ═══

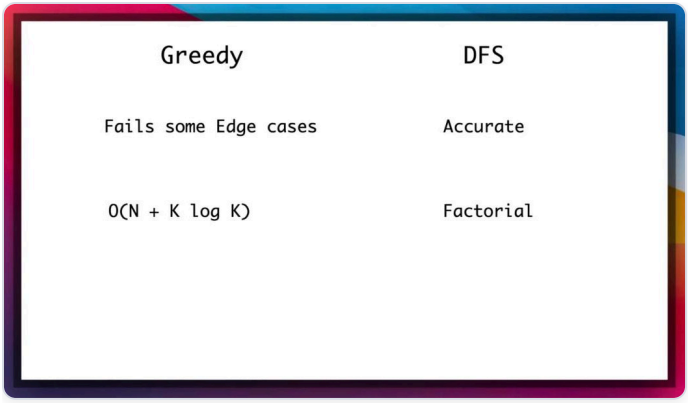
[Goa Trip] before: 2 txns  Bob→Shubh 50, Tom→Bob 50
[Goa Trip] after:  1 txn   Tom→Shubh 50
[Goa Trip]
Balance sheet of Shubh
  Paid: 100.00  Expense: 50.00  Owe: 0.00  Get back: 50.00
    gets 50.00 from Tom
Balance sheet of Bob
  Paid: 100.00  Expense: 100.00  Owe: 0.00  Get back: 0.00
Balance sheet of Tom
  Paid: 0.00  Expense: 50.00  Owe: 50.00  Get back: 0.00
    owes 50.00 to Shubh

[App demo] before: 3 txns  Shubh→Parth 6, Jay→Shubh 60, Jay→Parth 6
[App demo] after:  2 txns  Jay→Shubh 54, Jay→Parth 12

[Graph] before: 7 txns  A→B 300, B→C 400, C→A 50, C→D 150, D→B 100, D→E 100, E→A 50
[Graph] after:  2 txns  A→C 200, D→E 50

[Greedy edge case] before: 4 txns  R→P 30, S→Q 20, T→Q 10, T→P 10
[Greedy edge case] after:  4 txns  R→P 30, S→P 10, S→Q 10, T→Q 20
  Optimal is 3: R → Q 30, S → P 20, T → P 20 (greedy cannot see the {Q,R} pair)
```

Greedy vs DFS (backtracking)



Video @ 43:45 · the trade-off the video chose.

The video picks greedy: *O(N + K log K)*, fine for thousands of users, but it "fails some edge cases". Demo 4 above shows one. Nets are P +40, Q +30, R −30, S −20, T −20. Greedy pairs P with R first and needs **4** payments. The optimum is **3** (R→Q 30, S→P 20, T→P 20), because {Q, R} is a *zero-sum subgroup* that can settle on its own.

The optimum equals K − (the largest number of disjoint zero-sum subgroups), and finding that is NP-hard in general. With more than ~20 people the search explodes, which is why the video settles for greedy.

For the curious: the exact answer by backtracking (not in the code folders)

```
// Exact minimum: LeetCode 465, O(K!) in the worst case.
int minPayments(double[] net, int i) {
    while (i < net.length && Math.abs(net[i]) < 1e-6) i++;
    if (i == net.length) return 0;
    int best = Integer.MAX_VALUE;
    for (int j = i + 1; j < net.length; j++) {
        if (net[i] * net[j] >= 0) continue; // same sign
        net[j] += net[i];                   // i settles with j
        best = Math.min(best, 1 + minPayments(net, i + 1));
        net[j] -= net[i];                   // backtrack
    }
    return best;
}
```

Simplify is destructive

Once `simplifyDebts` runs, the pairwise history is gone: in the Goa demo, Bob's sheet no longer says he owes Shubh. New expenses still work, because they add on top of correct net positions. But you can't "un-simplify". The real app keeps the raw ledger and computes the simplified view when it shows it. That's a good point to raise in an interview.

Checkpoint

1) Why can a simplified graph contain an edge (A→C) that never existed?
2) Prove greedy never needs more than K − 1 payments.
3) Which heap holds debtors, and why is it a *min*-heap?
4) Should the "Non-Group Expenses" group ever be simplified? *(No: those people may not know each other.)*

PHASE 6

Extensions

Open/Closed in practice · Money rounding · Validation · Settle up

Beyond the video: the follow-ups interviewers ask next. Add an EXACT (unequal) split without touching existing strategies, make equal splits add up to the cent, reject bad input, and record a cash settle-up.

Phase-6-Extensions/

`ExactSplitStrategy` `SplitType.EXACT` `settleUp / recordPayment` `validation` `EPS`

1. A new split type: the Open/Closed promise, kept

The extensible requirement says "we can add more criteria in future". Here is the full cost of adding EXACT:

- ✓ `SplitType`: one enum constant.
- ✓ `ExactSplitStrategy`: one new class.
- ✓ `SplitStrategyFactory`: one `case` (the compiler insists).
- ✓ Nothing else. `ExpenseService` and the other strategies would work unchanged. (Phase 6 does edit them, but for the separate rounding and validation fixes below.)

2. Money that adds up

100 / 3 = 33.333… in a `double`. Printed to cents that is 33.33 × 3 = 99.99. A penny vanished. The new `EqualSplitStrategy` splits whole cents and hands the leftover cents to the first participants: 33.34, 33.33, 33.33.

strategy/ExactSplitStrategy.java — NEW

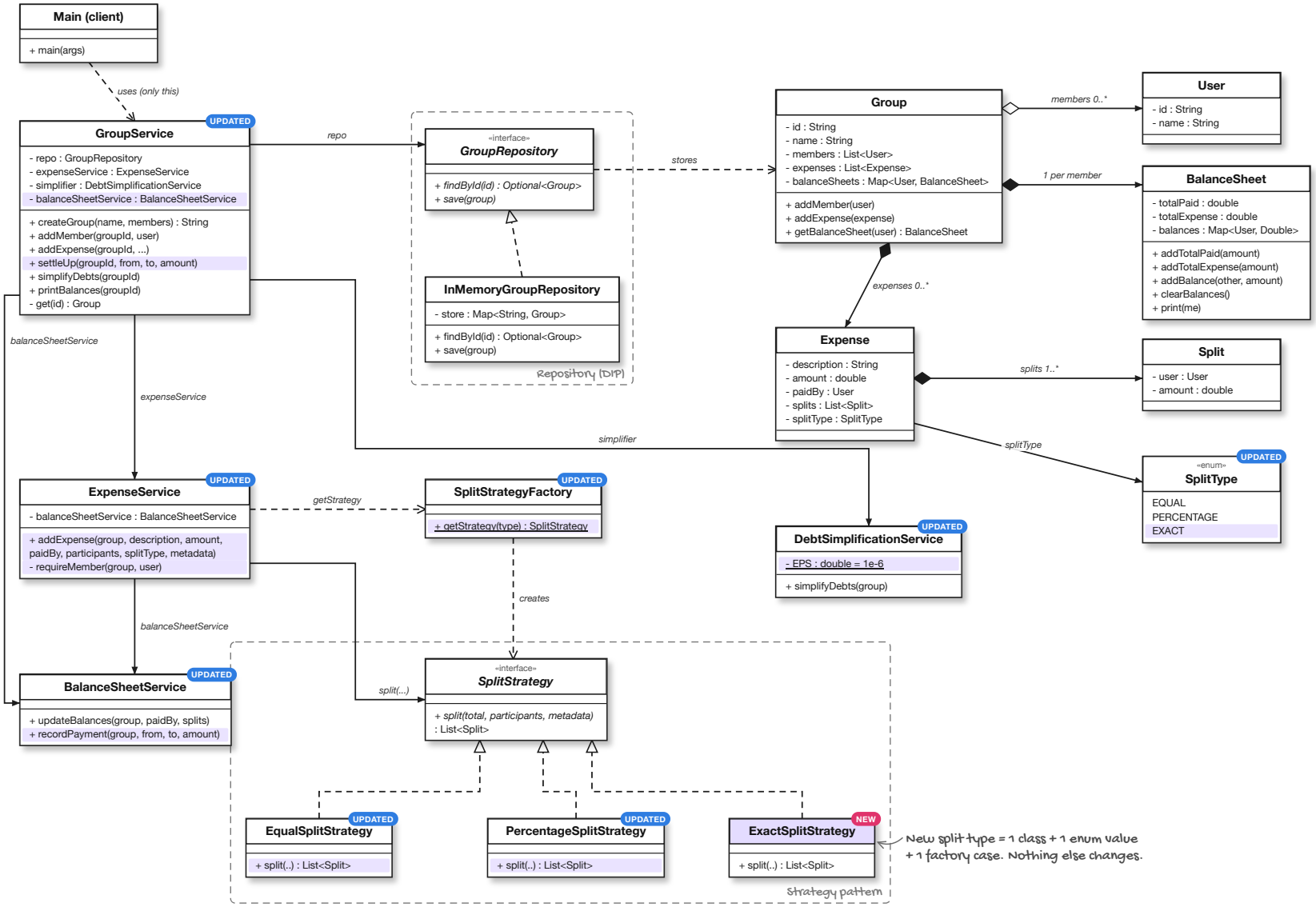
```java
// "Unequal" split: the caller states each person's exact
    share in metadata.
public class ExactSplitStrategy implements SplitStrategy {
    @Override
    public List<Split> split(double totalAmount, List<User>
     participants, Map<User, Double> metadata) {
        if (metadata == null ||
     !metadata.keySet().equals(new HashSet<>
     (participants)))
            throw new IllegalArgumentException("Give an
     amount for every participant, and only for them");
        double sum =
     metadata.values().stream().mapToDouble(Double::doubleV
     alue).sum();
        if (Math.abs(sum - totalAmount) > 0.005)
            throw new IllegalArgumentException("Exact
     amounts add up to " + sum + ", not " + totalAmount);

        List<Split> splits = new ArrayList<>();
        for (User user : participants) {
            splits.add(new Split(user,
     metadata.get(user)));
        }
        return splits;
    }
}
```

factory/SplitStrategyFactory.java — CHANGED

```java
public class SplitStrategyFactory {
    public static SplitStrategy getStrategy(SplitType
     splitType) {
        return switch (splitType) {
            case EQUAL -> new EqualSplitStrategy();
            case PERCENTAGE -> new
     PercentageSplitStrategy();
            case EXACT -> new ExactSplitStrategy();
        };
    }
}
```

Phase 6 UML: extensions without rewrites



New: `ExactSplitStrategy` and `SplitType.EXACT`. Shaded rows are the only members that changed; no class was restructured.

strategy/EqualSplitStrategy.java — CHANGED

```java
public class EqualSplitStrategy implements SplitStrategy {
    @Override
    public List<Split> split(double totalAmount, List<User> participants, Map<User, Double> metadata) {
        // Work in whole cents so the shares always add back up to the total:
        // 100.00 / 3 → 33.34, 33.33, 33.33 (the first `remainder` people pay one extra cent).
        long totalCents = Math.round(totalAmount * 100);
        long base = totalCents / participants.size();
        long remainder = totalCents % participants.size();

        List<Split> splits = new ArrayList<>();
        for (int i = 0; i < participants.size(); i++) {
            long cents = base + (i < remainder ? 1 : 0);
            splits.add(new Split(participants.get(i), cents / 100.0));
        }
        return splits;
    }
}
```

service/ExpenseService.java — CHANGED

```java
    public void addExpense(Group group, String description, double amount, User paidBy,
                           List<User> participants, SplitType splitType, Map<User, Double> metadata) {
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        if (participants.isEmpty()) throw new IllegalArgumentException("An expense needs participants");
        requireMember(group, paidBy);
        participants.forEach(u -> requireMember(group, u));

        SplitStrategy strategy = SplitStrategyFactory.getStrategy(splitType);
        List<Split> splits = strategy.split(amount, participants, metadata);

        // One guard for every strategy, present and future: the shares must cover the bill.
        double covered = splits.stream().mapToDouble(Split::getAmount).sum();
        if (Math.abs(covered - amount) > 0.005)
            throw new IllegalStateException(splitType + " split covers " + covered + " of " + amount);

        Expense expense = new Expense(description, amount, paidBy, splits, splitType);
        group.addExpense(expense);

        balanceSheetService.updateBalances(group, paidBy, splits);
    }
```

3. Validate once, at the orchestrator

`ExpenseService` now checks that the payer and participants belong to the group, and that whatever strategy ran produced shares that **cover the bill**. That one guard protects every future strategy too.

4. Settle up is not an expense

When Tom hands Shubh cash, nobody spent anything new, so `totalPaid` / `totalExpense` must not move. `recordPayment` only shifts the pairwise balance, and `settleUp` refuses to overpay a debt that doesn't exist.

service/BalanceSheetService.java (new method) — CHANGED

```java
    // A settle-up is money moving between two people, not an expense: it changes who owes
    // whom, but nobody's totalPaid / totalExpense (what the group spent) changes.
    public void recordPayment(Group group, User from, User to, double amount) {
        group.getBalanceSheet(from).addBalance(to, amount);
        group.getBalanceSheet(to).addBalance(from, -amount);
    }
```

service/GroupService.java (new method) CHANGED

```java
    public void settleUp(String groupId, User from, User to, double amount) {
        Group g = get(groupId);
        double owed = Math.max(0, -g.getBalanceSheet(from).getBalances().getOrDefault(to, 0.0));
        if (amount ⩽ 0 || amount > owed + 0.005)
            throw new IllegalArgumentException(String.format("%s owes %s only %.2f", from.getName(), to.getName(), owed));
        balanceSheetService.recordPayment(g, from, to, amount);
    }
```

A precise note on the percentage check

The reference compares `totalPercent ≠ 100.0`. In practice that works, because `DoubleStream.sum()` uses compensated summation. A plain `+=` loop over 40.7 + 40.6 + 18.7 gives 100.00000000000001, though. Phase 6 compares with a tolerance and also rejects percentages given for non-participants. That second case would otherwise silently leave part of the bill unpaid.

```
▶ bash Phase-6-Extensions/run.sh

═══ Phase 6: Extensions (exact split, settle up, money rounding, validation) ═══
[Goa Trip]
Balance sheet of Shubh
  Paid: 100.00  Expense: 133.34  Owe: 66.67  Get back: 33.33
    owes 66.67 to Bob
    gets 33.33 from Tom
Balance sheet of Bob
  Paid: 250.00  Expense: 143.33  Owe: 0.00  Get back: 106.67
    gets 66.67 from Shubh
    gets 40.00 from Tom
Balance sheet of Tom
  Paid: 120.00  Expense: 193.33  Owe: 73.33  Get back: 0.00
    owes 33.33 to Shubh
    owes 40.00 to Bob

Tom pays Shubh 33.33 (settle up)
[Goa Trip]
Balance sheet of Shubh
  Paid: 100.00  Expense: 133.34  Owe: 66.67  Get back: 0.00
    owes 66.67 to Bob
Balance sheet of Bob
  Paid: 250.00  Expense: 143.33  Owe: 0.00  Get back: 106.67
    gets 66.67 from Shubh
    gets 40.00 from Tom
Balance sheet of Tom
  Paid: 120.00  Expense: 193.33  Owe: 40.00  Get back: 0.00
    owes 40.00 to Bob

  Jay is not in the group → rejected: Jay is not in Goa Trip
  Exact amounts do not add up → rejected: Exact amounts add up to 80.0, not 90.0
  Percent given for a non-participant → rejected: Give a percentage for every participant, and only for them
  Overpaying a settle up → rejected: Tom owes Bob only 40.00
  Settling a debt that does not exist → rejected: Bob owes Shubh only 0.00
```

Check it by hand: expenses total 100 + 250 + 120 = 470, and the three *Expense* lines add up to 133.34 + 143.33 + 193.33 = 470. The settle-up removes Tom's 33.33 debt to Shubh but leaves every total unchanged.

Still a double

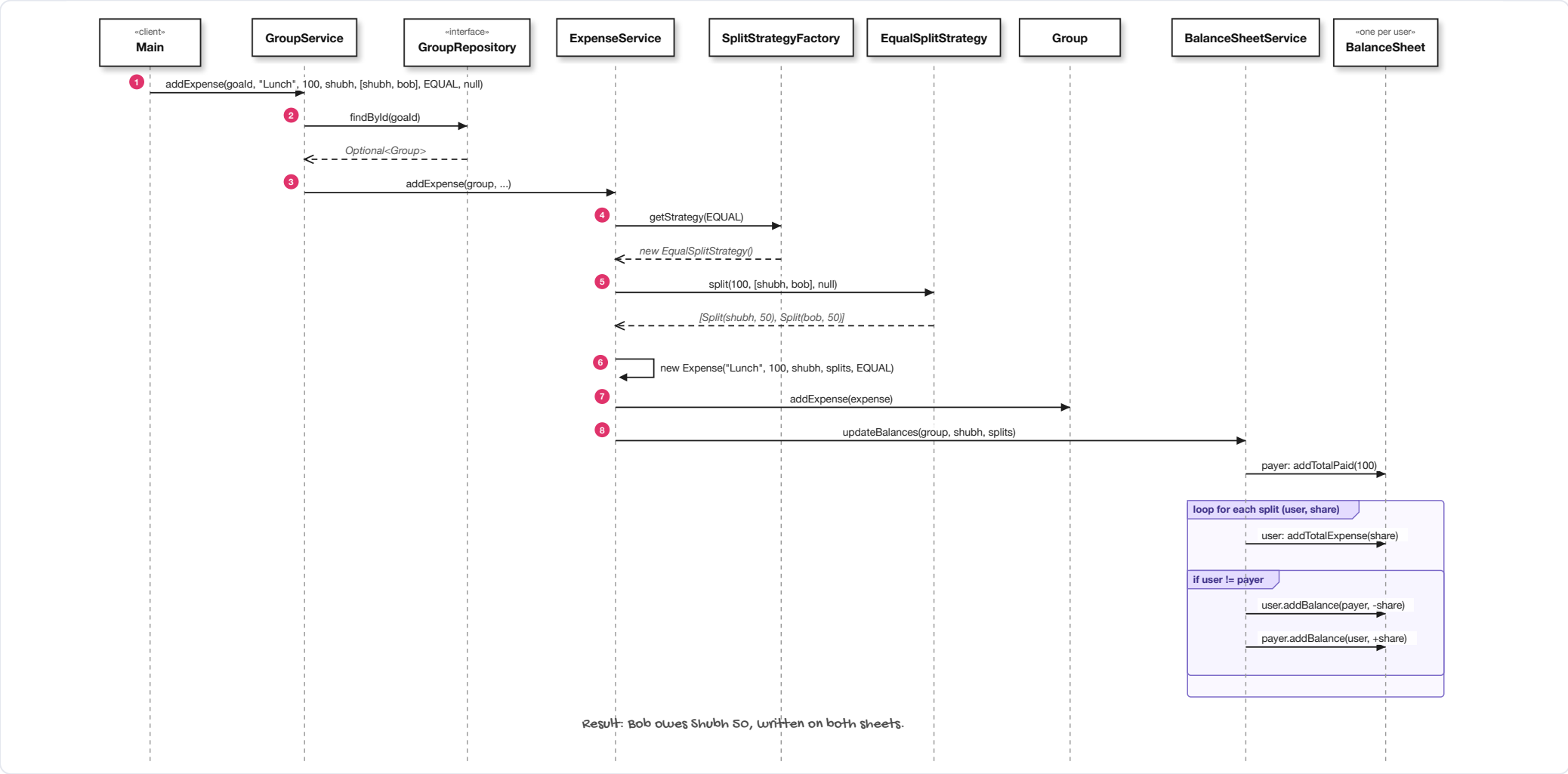
Cent-splitting fixes the visible penny, but production code should not hold money in `double` at all. Use `long` cents or `BigDecimal` with a currency. Say so in the interview, even if you code with `double` for speed.

Checkpoint

1) Add a SHARES split ("Shubh 2 shares, Bob 1"). List every file you touch.
2) Why must `recordPayment` not call `addTotalPaid`?
3) Where would you put a rule like "max 50 participants per expense"? Why there?

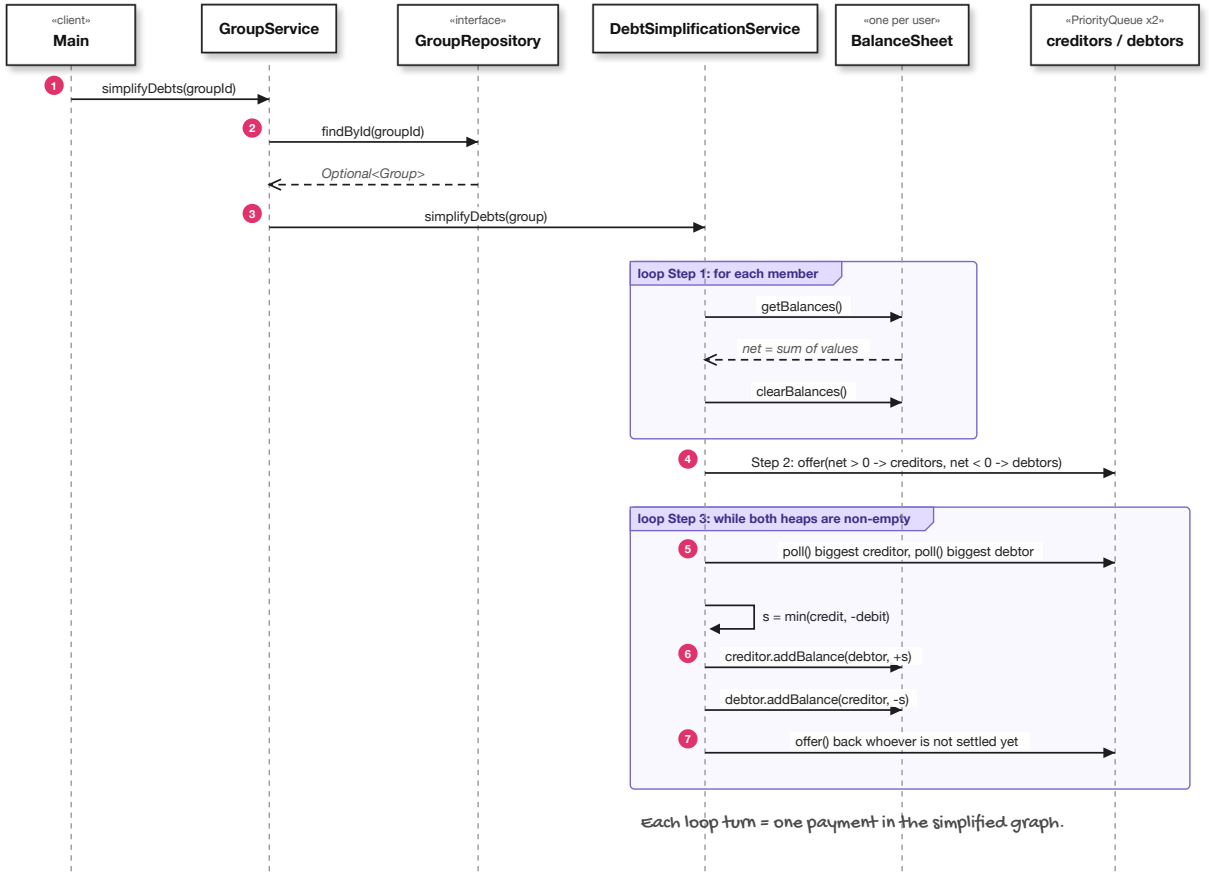
Putting it together: the complete code flow

Flow 1: `addExpense`



1 The client only knows the group id and calls the facade. **2** `GroupService` loads the group through the repository interface (unknown id → `IllegalArgumentException`). **3** It delegates to `ExpenseService`. **4** The factory turns `EQUAL` into an `EqualSplitStrategy`. **5** The strategy returns the splits. **6–7** An immutable `Expense` is created and stored on the group. **8** `BalanceSheetService` writes the payer's total paid, each participant's total expense, and both sides of each debt.

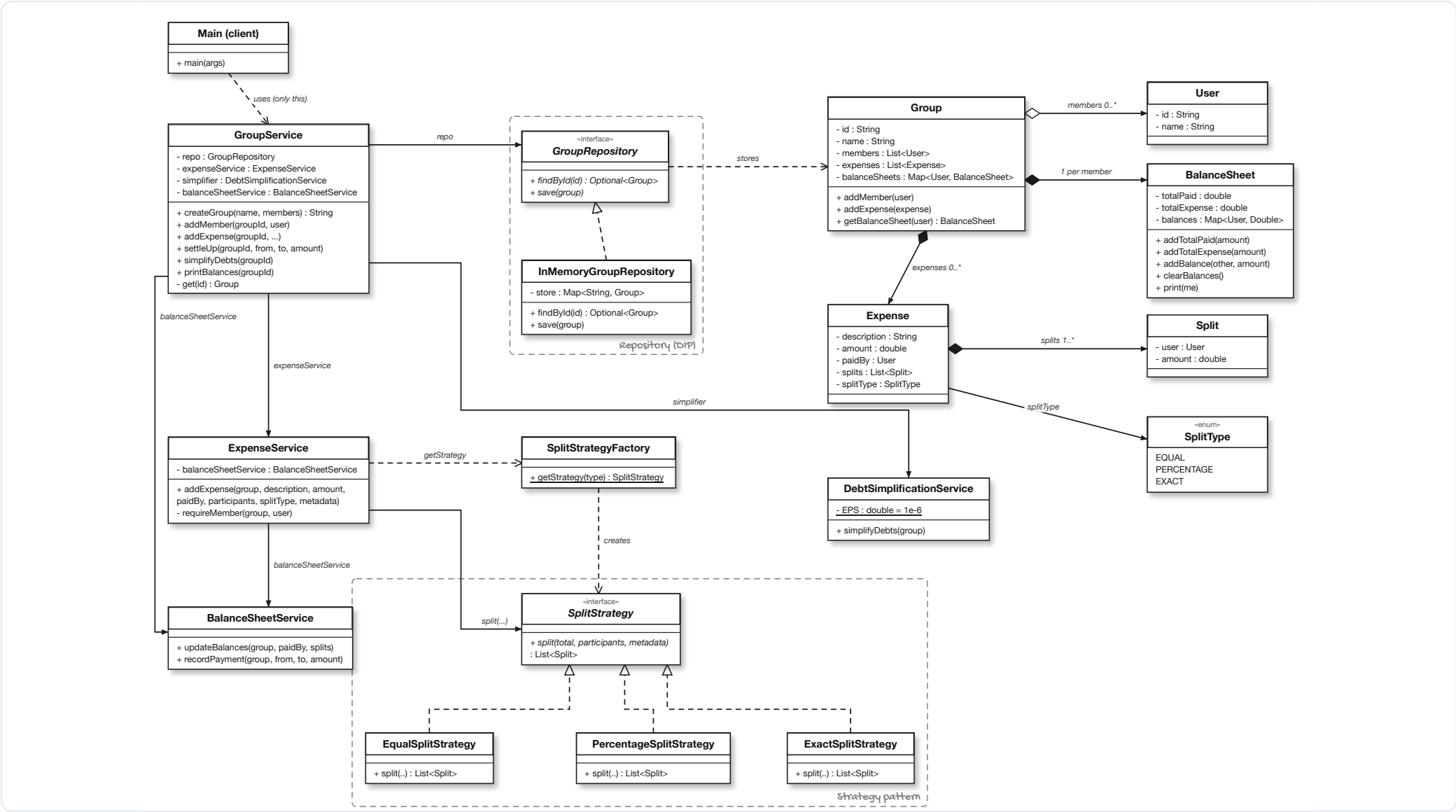
Flow 2: `simplifyDebts`



> **Explain it in 30 seconds**
>
> "Users belong to groups; a 1-on-1 is a hidden default group. An expense has a payer and a list of *splits*, produced by a **Strategy** chosen through a **factory** from the split type. Every expense updates each member's per-group **balance sheet** on both sides. Clients only use **GroupService**, a facade that loads groups through a **repository interface**. Simplify debts collapses everyone to a net balance and greedily matches the largest creditor with the largest debtor using two heaps, so K people need at most K − 1 payments."

1–3 Same entry path as Flow 1, ending in `DebtSimplificationService`. **Step 1**: for every member, sum the balances map to get a net, then clear the map. **4** Nets > 0 go into the creditor max-heap, nets < 0 into the debtor min-heap. **5–7** Each loop turn polls the biggest of each, settles the smaller amount, writes that one debt on both sheets, and re-offers whoever is not yet at zero.

The complete design (all six phases stitched together)



Phases 1–5 are the video's design. Phase 6 adds `ExactSplitStrategy`, `SplitType.EXACT`, `settleUp` / `recordPayment`, validation in `ExpenseService` and an epsilon in the simplifier. The same diagram is in `diagrams/complete-class-diagram.svg`, which you can zoom freely.

Notes on the reference code

Phase 5 has the same 17 classes and the same algorithms as the reference repository. These are the deliberate differences:

Reference	In these notes	Why
Lombok `@Getter`, `@AllArgsConstructor`, `@ToString`	Plain constructors and getters	Compiles with just `javac`; nothing hidden from the reader
Package `org.nailyourinterview.lld.splitwise`	`splitwise`	Shorter paths in the notes
`balances` is a `HashMap`	`LinkedHashMap`	Users have identity hash codes, so a HashMap prints in a different order every run
`GroupService.printBalances` formats its own output	Delegates to `BalanceSheet.print`	The reference had two printers; now there is one
Main creates a "Non-Group Expenses" group but never uses it	Phase 4 uses it for the video's 1-on-1 example	Shows pairwise netting (Shubh owes Jay 5)
Whiteboard: `SplitStrategyFactory` «interface», `BalanceSheetService.printGroupBalances`, `GroupRepo.groups`	Factory is a class with a static method; no `printGroupBalances`; the map lives in `InMemoryGroupRepository.store`	Diagrams follow the **code** where the whiteboard and code differ

Where to take it next

Concurrency

Two `addExpense` calls on one group race on `addBalance` (read-modify-write on a `HashMap`). Lock per group (one `ReentrantLock` per group id), or use `ConcurrentHashMap.merge` on the balances.

Edit / delete an expense

Replay its splits with negated amounts through `updateBalances`. That fails after a simplify, which is one more reason to keep the raw ledger and simplify only for display.

Notifications (Observer)

Publish `ExpenseAdded` / `DebtSettled` events from `GroupService`; email, push and activity-feed listeners subscribe without touching the services.

Persistence

`SqlGroupRepository implements GroupRepository`. Give `User` `equals` / `hashCode` on id first, or the balance maps break on reload.

Money & currency

A `Money(long cents, Currency)` value object, conversion at expense time, and balances kept per currency.

More split types

SHARES, ADJUSTMENT ("equal, but Bob +5"). Each one is a single `SplitStrategy` class. Try one as an exercise.

Self-test: interview questions

Cover the grey answers, say yours out loud, then check.

1. **Why does `Expense` hold `List<Split>` instead of `List<User>`?**
   Unequal splits need who *and* how much; a Split carries both, and the users are derivable.
2. **Which pattern handles split types, and what does the factory add?**
   Strategy encapsulates each algorithm behind one interface; the Simple Factory maps the runtime `SplitType` to the object, keeping the single `switch` in one place.
3. **What changes to add a SHARES split?**
   One enum value, one strategy class, one factory case. Nothing else (OCP).
4. **What three things are in a balance sheet, and what invariant ties them together?**
   totalPaid, totalExpense, balances; totalPaid − totalExpense = Σ balances, and Σ over the group = 0.
5. **Why is a 1-on-1 expense modelled as a group?**
   Same code path; the default group is just never simplified group-wide.
6. **How does DIP show up here, and what does it buy you?**
   `GroupService` depends on the `GroupRepository` interface, so you can swap storage or inject a fake in tests without editing services.
7. **Walk through `simplifyDebts`.**
   Nets per user → clear sheets → creditor max-heap / debtor min-heap → repeatedly settle min(credit, −debit) and re-offer → write each payment on both sheets.
8. **Complexity of the greedy, and its upper bound on payments?**
   O(N + E + K log K); at most K − 1 payments for K non-zero users.
9. **Give a case where greedy is not optimal.**
   Nets +40, +30, −30, −20, −20: greedy needs 4, optimal 3 (the zero-sum pair +30/−30).
10. **Why does settle-up not change totalPaid?**
    It moves existing debt, it is not new spending; totals describe consumption.
11. **What breaks if two threads add expenses to one group at once?**
    Lost updates in `addBalance` (unsynchronised read-modify-write). Lock per group or use atomic merge.
12. **Why is `double` a bad type for money, and what do you use instead?**
    Binary fractions can't represent cents exactly, so sums drift. Use long cents or BigDecimal + currency.

Final drill: draw the complete diagram from a blank page, then build Phase 5 from an empty folder and run it. ✎